

Trabajo de Fin de Máster

Evaluación de LLMs para la Generación de Ontologías en Bases de Datos Genéticas

Máster Universitario en Inteligencia Artificial

Facultad de Informática · Universidad de Murcia

Documento de metodología

Autor: Francisco Sáez

Versión: 11.0

Fecha: 9 de mayo de 2026

Tutor: [Nombre del tutor o tutora]

Resumen ejecutivo

Construir una ontología biomédica a mano lleva semanas. Dos ontólogos rara vez convergen en el mismo esquema. La pregunta de este TFM es operativa: dado un CSV de una base de datos genética, ¿qué tan lejos puede llegar un LLM solo, y dónde merece la pena ayudarlo?

Para responderlo construyo un banco de pruebas con cuatro estrategias (zero-shot, vocabulario controlado, RAG, pipeline multi-agente OntoGenix), tres modelos (gpt-4o, Llama 3.1 8B, Qwen 2.5 Coder 7B) y cuatro bases de datos del dominio cis-regulatorio (FANTOM5, dbSUPER, HACER, DiseaseEnhancer). Genero unas 140 ontologías y las evalúo con OQuaRE, el marco normativo basado en ISO/IEC 25000, integrando el razonador HermiT a través de owlready2. Sustituyo además el RAG por keywords del experimento original por uno semántico real, indexado en Neo4j con embeddings de sentence-transformers.

Los resultados son menos lineales de lo que esperaba. La pipeline multi-agente E4 (OntoGenix) gana en riqueza absoluta —ontologías entre 3.7× y 5.3× más densas en triples que las baselines E1-E3 (media 4.4×)— pero pierde en OQuaRE, porque las rúbricas penalizan la complejidad estructural. El RAG semántico real eleva el score E3 de 2.91 a 4.20 en gpt-4o, un salto de +44 %, gracias a una mejora ×6 en operatividad relacional. Cuando aplico el mismo RAG a Llama, el efecto se invierte: el modelo pequeño se satura con el contexto rico y la mitad de las ejecuciones dejan de validar. Bajar top_k a 2 cierra el 84 % del gap respecto a gpt-4o sin tocar el modelo. Cada estrategia tiene su patrón de error característico: Llama omite el «:» de los prefijos, Qwen mezcla sintaxis Turtle con RDF/XML, gpt-4o no falla.

El TFM aporta tres contribuciones que considero replicables. Una guía empírica de calibración del RAG por tamaño de modelo (§10.7.7). Una serie de fixes mecánicos que rescatan el 100 % de los TTL recuperables sin volver a llamar al modelo (§7). Un marco de reproducibilidad alineado con la crisis de la replicación (§12) que reconoce explícitamente lo que los LLM hacen difícil de garantizar. Lo que queda fuera es igual de relevante: ningún experimento aquí prueba que un LLM pueda sustituir a un ontólogo en producción, y los benchmarks normativos como OQuaRE pueden favorecer ontologías pobres si no se interpretan junto a métricas absolutas de cobertura.

Abstract

Building a biomedical ontology by hand takes weeks. Two ontologists rarely converge on the same schema. The operational question of this Master's Thesis is: given a CSV from a genetic database, how far can a Large Language Model go on its own, and where is it worth helping it?

I build a benchmark with four strategies (zero-shot, controlled vocabulary, RAG, multi-agent OntoGenix pipeline), three models (gpt-4o, Llama 3.1 8B, Qwen 2.5 Coder 7B) and four cis-regulatory databases (FANTOM5, dbSUPER, HACER, DiseaseEnhancer). I generate around 140 ontologies and evaluate them with OQuaRE — the normative framework based on ISO/IEC 25000 — integrating the HermiT reasoner via owlready2. I also replace the keyword-RAG with a real semantic RAG, indexed in Neo4j with sentence-transformers embeddings.

The multi-agent E4 pipeline (OntoGenix) wins in absolute richness — ontologies between 3.7× and 5.3× denser than E1–E3 baselines (mean 4.4×) — but loses on OQuaRE because the rubric penalises structural complexity. The semantic RAG raises gpt-4o's E3 score from 2.91 to 4.20, a +44% gain driven by a ×6 improvement in relational operability. When the same RAG is applied to Llama, the effect inverts: the small model saturates with rich context and half the runs no longer validate. Lowering top_k to 2 closes 84% of the gap relative to gpt-4o without changing the model. Each strategy has its characteristic error pattern: Llama omits the prefix colon, Qwen mixes Turtle with RDF/XML, gpt-4o does not fail.

The thesis contributes three replicable findings: an empirical calibration guide for RAG by model size, a set of mechanical fixes that rescue 100% of recoverable TTLs without re-prompting, and a reproducibility framework aligned with the replication crisis in machine learning research.

Palabras clave / Keywords

Palabras clave: ontologías biomédicas; modelos de lenguaje grandes; generación de ontologías; RAG semántico; OQuaRE; OntoGenix; reproducibilidad; dominio cis-regulatorio; OWL; HermiT.

Keywords: biomedical ontologies; Large Language Models; ontology generation; semantic Retrieval-Augmented Generation; OQuaRE; OntoGenix; reproducibility; cis-regulatory domain; OWL; HermiT.

Lista de tablas

Tabla 1. Configuración global del experimento (campo y valor por defecto)

Tabla 2. Pipeline replicada de OntoGenix, paso a paso

Tabla 3. Adaptaciones técnicas de OntoGenix al entorno del TFM

Tabla 4. Parámetros de generación comunes (E1–E3)

Tabla 5. Corpus de datos del dominio cis-regulatorio utilizado

Tabla 6. Estrategias de muestreo evaluadas (cabecera, aleatorio, estratificado, diversidad)

Tabla 7. Resultados del análisis de sensibilidad por (modelo × experimento × estrategia)

Tabla 8. Roles de los tres agentes de la pipeline OntoGenix

Tabla 9. Métricas estructurales de evaluación calculadas sobre cada ontología

Tabla 10. Efecto del post-procesado mecánico sobre la validez sintáctica

Tabla 11. Tabla resumen consolidada — métricas absolutas por experimento y base de datos

Tabla 12. Rúbricas OQuaRE (Duque-Ramos et al., 2014) por sub-característica

Tabla 13. OQuaRE Global por modelo y experimento (escala 1–5)

Tabla 14. Validez sintáctica cross-model tras post-procesado

Tabla 15. OQuaRE Global por celda — modelo × experimento × base de datos

Tabla 16. Comparativa E4 OntoGenix con gpt-4o frente a Llama 3.1 8B (riqueza vs sintaxis)

Tabla 17. Arquitectura del RAG semántico (annotationRAG): componentes y responsabilidades

Tabla 18. Descomposición OQuaRE: RAG por keywords frente a RAG semántico (gpt-4o)

Tabla 19. Coste de la integración del RAG semántico (recursos y dependencias)

Tabla 20. Cuadro 2×2: efecto del backend RAG (legacy vs semántico) en gpt-4o y Llama 3.1 8B

Tabla 21. Configuraciones de calibración del RAG (C1: top_k=2, C2: thr=0.6, C3: combinada)

Tabla 22. Resultados de la calibración del RAG semántico sobre Llama 3.1 8B en E3

Tabla 23. Recomendación operativa de top_k según el tamaño del modelo

Tabla 24. Pila de herramientas y dependencias del banco experimental

Tabla 25. Amenazas específicas a la reproducibilidad en investigación con LLMs

Tabla 26. Tres niveles de garantía de reproducibilidad (R1 estricto · R2 estadístico · R3 metodológico)

1. Introducción y objetivos

Las ontologías biomédicas son caras. Formalizar la jerarquía de elementos cis-regulatorios humanos (enhancers, super-enhancers, elementos reguladores activos) requiere semanas de trabajo de un experto en semántica que también conozca el dominio. Hay pocos perfiles así, y los que existen están saturados. Las bases de datos del dominio crecen más rápido que la capacidad de los ontólogos para mantenerlas anotadas.

Los modelos de lenguaje grandes producen Turtle. Lo hacen bien, lo hacen mal o lo hacen a medias según cómo se les pregunte. La literatura propone cuatro estrategias principales: zero-shot, vocabulario controlado, RAG y pipelines multi-agente. La más elaborada (OntoGenix, Val Calvo et al., 2024) usa tres agentes secuenciales y promete ontologías más ricas que cualquier prompt directo. Las afirmaciones de la literatura, sin embargo, son difíciles de comparar porque cada paper usa su propio dataset, su propia métrica y su propio modelo cerrado.

El TFM mide las cuatro estrategias en el mismo banco. Mismas bases de datos, mismas semillas, mismas métricas, los mismos tres modelos. La pregunta de fondo no es académica: si un equipo biomédico tiene 40 horas para anotar una nueva fuente de datos, ¿qué combinación de modelo y

técnica le da el mejor punto de partida? La respuesta no es una sola, depende de qué tipo de error pueda permitirse. Lo que sigue es la base de evidencia que permite hacer esa elección con criterio.

1.1. Objetivos específicos (alcanzados)

Los objetivos iniciales del TFM, planteados en la versión 1.0 de esta memoria, eran cuatro. La versión actual los amplía con dos objetivos emergentes que surgieron durante la ejecución y que han terminado siendo tan relevantes como los de partida. Lista completa, marcando cuáles eran iniciales (I) y cuáles emergieron durante el trabajo (E):

- **(I) Banco de pruebas reproducible.** Construido con cuatro experimentos (E1–E4), cuatro bases de datos del dominio cisreg (FANTOM5, dbSUPER, HACER, DiseaseEnhancer) y tres semillas (42, 43, 44) para estimar variabilidad inter-ejecución. 96 ejecuciones en el experimento principal y 144 ejecuciones adicionales en los sub-experimentos.
- **(I) Replicación funcional de OntoGenix.** Pipeline ejecutada sobre las cuatro BBDD sin la GUI PyQt5, con tres adaptaciones técnicas documentadas (bypass de Searcher, bypass del paso SERP, snapshot del modelo). Detalle en §3.
- **(I) Métricas objetivas, no caseras.** OQuaRE como marco normativo (§9), implementado contra el razonador HermiT vía owlready2. Cinco sub-características más Global, todas en escala 1–5 con rúbricas publicadas (Duque-Ramos et al., 2014).
- **(I) Patrones de error por estrategia y por modelo** documentados en §10.2: Llama omite el «:» del prefijo en cuerpos del esquema; Qwen mezcla sintaxis Turtle con RDF/XML y JSON-LD; gpt-4o no comete ninguno de los dos. Las plantillas cisreg originales también tenían erratas tipográficas (un «>» espurio en crm2tfac.ttl) que el experimento sacó a la luz.
- **(E) Comparativa cross-model** (§10) entre modelo cerrado (gpt-4o) y dos open-source (Llama 3.1 8B, Qwen 2.5 Coder 7B). Resultado contraintuitivo: en E2 el modelo pequeño supera al grande en OQuaRE, porque la rúbrica penaliza la complejidad. La comparación cualitativa de §8 sigue dando ventaja a gpt-4o en riqueza absoluta.
- **(E) RAG semántico real** sustituye al keyword-match estático del experimento original (§10.7). Subo el OQuaRE de E3 con gpt-4o de 2.91 a 4.20 (+44 %). Para Llama el efecto es opuesto hasta calibrar top_k=2, momento en el que el 84 % del gap modelo-cerrado se cierra (§10.7.7). La calibración constituye una guía operativa cuantitativa, no una recomendación cualitativa.

1.2. Lo que NO cubre este TFM

Reconocer el alcance es tan importante como reportar los logros. Cuatro cosas que el trabajo no demuestra y que merecen aclararse:

- **No prueba que un LLM pueda sustituir a un ontólogo en producción.** Las ontologías generadas necesitan revisión humana para uso real. El TFM mide cuánto trabajo ahorra el LLM, no si el resultado es publicable.

- **No es generalizable a otros dominios sin replicación.** Todos los experimentos usan bases de datos cis-regulatorias humanas. Un dominio con vocabulario más amplio (clinical trials, drug-target) podría dar otra distribución de errores y otro ranking entre modelos.
- **No evalúa la calidad de las relaciones inferidas.** OQuaRE y sus 11 métricas estructurales miden la forma del esquema, no la corrección biomédica de las clases que declara. Para eso haría falta validación experta sobre una muestra.
- **No es 100 % reproducible bit-a-bit.** OpenAI no garantiza determinismo, aunque la temperatura sea 0 y la seed esté fijada. El TFM admite la limitación y la encuadra en el marco de reproducibilidad de §12.

2. Marco teórico

2.1. Ontologías y OWL 2

Una ontología es una especificación formal y explícita de una conceptualización compartida (Studer, Benjamins & Fensel, 1998). En la web semántica se expresan habitualmente en OWL 2 serializado en Turtle (W3C, 2012), una sintaxis orientada a triples sujeto-predicado-objeto. Los elementos fundamentales son las clases (owl:Class), las propiedades de objeto (owl:ObjectProperty), las propiedades de datos (owl:DatatypeProperty), las restricciones (owl:Restriction) y los axiomas subsumptivos (rdfs:subClassOf).

2.2. Generación de ontologías con LLM

El uso de LLM para tareas de ingeniería ontológica ha generado en los últimos dos años una línea de trabajo activa. Tres estrategias emergen como referentes: la generación zero-shot a partir de texto o datos en bruto, la generación guiada por vocabularios controlados, y la generación aumentada por recuperación (RAG) con fragmentos de ontologías de referencia (Meyer et al., 2023; Neuhaus, 2023). A estas se suma una cuarta aproximación más reciente: las pipelines multi-agente, donde un LLM planifica, otro escribe el esquema y un tercero lo mapea contra datos. OntoGenix es el exponente publicado más completo de esta aproximación (Val Calvo et al., 2024).

3. Artículo de referencia: OntoGenix

La herramienta sobre la que se apoya el experimento E4 del presente trabajo es OntoGenix, publicada por el grupo Tecnomod de la Universidad de Murcia. Los datos de cita son:

Tabla 1. Configuración global del experimento (campo y valor por defecto)

Campo	Valor
Título	OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets
Autores	Val Calvo, M.; Egaña Aranguren, M.; Fernández Breis, J. T. (Universidad de Murcia · Universidad del País Vasco)

Repositorio	https://github.com/tecnomod-um/OntoGenix
Licencia	MIT (según LICENSE del repositorio)
Modelo LLM utilizado	gpt-4o-2024-05-13 (OpenAI)
Arquitectura	Sistema multi-agente con tres agentes LLM: LlmPlanner, LlmOntology y LlmOntoMapper

3.1. Pipeline replicada

OntoGenix implementa una pipeline de cinco pasos que se ha replicado íntegramente en el experimento E4, omitiendo únicamente el paso de interoperabilidad con SERP API por no disponer de clave. La tabla siguiente resume el flujo:

Tabla 2. Pipeline replicada de OntoGenix, paso a paso

Paso	Agente / función	Entrada	Salida
1	csv_data_preprocessing	CSV limpio de la base de datos	DataFrame con estadísticas por columna
2	dataframe2prettyjson	DataFrame de estadísticas	Descripción columnar en JSON
3	LlmPlanner.interaction()	JSON + descripción de la tarea	data_description_run{N}.md (razonamiento)
4	LlmOntology.interact()	JSON + descripción del planner	ontology_run{N}.ttl (OWL 2 Turtle)
5	LlmOntoMapper.interact()	Descripción + ontología + nombre del CSV	mapping_run{N}.ttl (mapping RML/TTL)

3.2. Adaptación al entorno del TFM

Para poder ejecutar OntoGenix de forma reproducible se han realizado las adaptaciones siguientes. Todas ellas quedan registradas en scripts/run_ontogenix_experiments.py dentro del repositorio del TFM.

- Bypass de la GUI PyQt5: se instancian directamente los agentes LlmPlanner, LlmOntology y LlmOntoMapper, reproduciendo en código la interpolación de plantillas que realiza el GuiManager a partir de metadata.json.
- Requirements mínimo: se genera scripts/requirements_e4.txt con solo openai, pandas, numpy, chardet, python-dotenv, python-dateutil, serpapi y rdflib. Se excluyen PyQt5 y morph-kgc por no ser necesarios en modo headless.
- Semilla y temperatura: temperatura = 0, seed base = 42 e incremento por run (42, 43, 44) para obtener repeticiones reproducibles.
- Omisión de LlmPlanner.update(): este paso invoca SERP API para añadir contexto de schema.org; se sustituye por una clave dummy que evita el crash del Searcher y no se ejecuta la llamada externa.

4. Diseño experimental

Se definen cuatro experimentos que comparten el mismo modelo base (gpt-4o) y el mismo corpus de datos, pero difieren en la estrategia de prompting y en la arquitectura del sistema. La comparación es, por tanto, apples-to-apples: cualquier diferencia observada es atribuible a la estrategia, no al modelo.

Tabla 3. Adaptaciones técnicas de OntoGenix al entorno del TFM

ID	Nombre	Estrategia	Contexto adicional
E1	Base (zero-shot)	Prompting directo	Solo la muestra de datos
E2	Vocabulario controlado	Prompting con vocabulario	Términos skos:prefLabel/ skos:definition de la ontología cisreg
E3	RAG ontología cisreg	Retrieval augmented	Fragmentos relevantes de la ontología cisreg (crm, crm2gene, crm2phen, crm2tfac)
E4	OntoGenix (multi- agente)	Pipeline multi-agente	Descripción generada por LlmPlanner + estadísticas del CSV

4.1. Parámetros de generación comunes

Tabla 4. Parámetros de generación comunes (E1–E3)

Parámetro	Valor	Justificación
Modelo	gpt-4o (OpenAI)	Modelo declarado por OntoGenix; se usa también en E1–E3 para garantizar
temperature	0.1 (E1–E3) / 0 (E4)	Se minimiza la aleatoriedad para favorecer reproducibilidad del código Turtle
top_p	0.9	Muestreo nucleico estándar
max_tokens	4096	Suficiente para ontologías de ≈200–300 triples
seed base	42	Reproducibilidad; cada run incrementa en +1 (42, 43, 44)
repeticiones	3 por (experimento, BBDD)	Estimación de variabilidad entre ejecuciones

4.2. Corpus de datos

Se emplea un subconjunto representativo de cuatro bases de datos del dominio cis-regulatorio. Se eligen porque cubren tipologías complementarias (enhancers activos, super-enhancers, elementos

regulatorios activos y enhancers asociados a enfermedad) y porque disponen de esquema tabular con metadatos experimentales claros.

Tabla 5. Corpus de datos del dominio cis-regulatorio utilizado

Base de datos	Descripción	Fuente
FANTOM5	Enhancers activos identificados por CAGE-seq	fantom.gsc.riken.jp/5/
dbSUPER	Super-enhancers con coordenadas, líneas celulares y genes diana	asntech.org/dbsuper/
HACER	Human Active Cis-regulatory Elements database	bioinfo.vanderbilt.edu/AE/HACER/
DiseaseEnhancer	Enhancers asociados a enfermedades humanas	biocc.hrbmu.edu.cn/DiseaseEnhancer/

Cada base se muestrea con un protocolo común: 5 filas de cabecera + 20 filas adicionales + límite de 3.000 tokens. Las muestras se almacenan en `data/samples/{DB}_sample_prompt.txt` y, para E4, se convierten a CSV en `data/csv_for_ontogenix/{DB}.csv`.

4.3. Análisis de sensibilidad al muestreo

El protocolo de muestreo descrito en §4.2 (cabecera: primeras 5+20 filas) es reproducible y determinista, pero podría introducir un sesgo si las bases de datos están ordenadas por algún campo categórico (cromosoma, línea celular, etc.). Para cuantificar la robustez de los resultados ante la elección de muestreo, se diseñó un sub-experimento que compara cuatro estrategias aplicadas a las dos bases más representativas (FANTOM5 con 65 423 enhancers y dbSUPER con 69 205 super-enhancers).

4.3.1. Estrategias evaluadas

Tabla 6. Estrategias de muestreo evaluadas (cabecera, aleatorio, estratificado, diversidad)

ID	Estrategia	Justificación	Cobertura categórica*
A	head — primeras N filas	Reproducible, determinista, baseline del TFM	FANTOM5: 4% chrom · dbSUPER: 1% cell_name
B	random — uniforme con seed=42	Estimador insesgado de la distribución del universo	FANTOM5: 62% chrom · dbSUPER: 22% cell_name
C	stratified — proporcional por cell_name/chrom	Garantiza representación de cada categoría observada	FANTOM5: 100% chrom · dbSUPER: 24% cell_name
D	diversity — máxima cobertura léxica greedy	Selección iterativa que añade el máximo de valores nuevos	FANTOM5: 100% chrom · dbSUPER: 24% cell_name

* Porcentaje de valores únicos del universo cubierto por la muestra de 25 filas en la columna categórica más informativa.

4.3.2. Diseño experimental

Se replicaron los experimentos E1 (zero-shot) y E4 (OntoGenix) con gpt-4o sobre cada (estrategia, BBDD), y E1 también con llama3.1:8b para verificar si el comportamiento del modelo open-source es estable. Total: 24 ejecuciones. Los resultados se evaluaron con el mismo pipeline OQuaRE (§9). Se eligió el esquema simple del TSV original (7-12 columnas) en lugar del esquema CRM enriquecido (30 columnas) usado en el experimento principal, para que la comparación inter-estrategia sea apples-to-apples.

4.3.3. Resultados

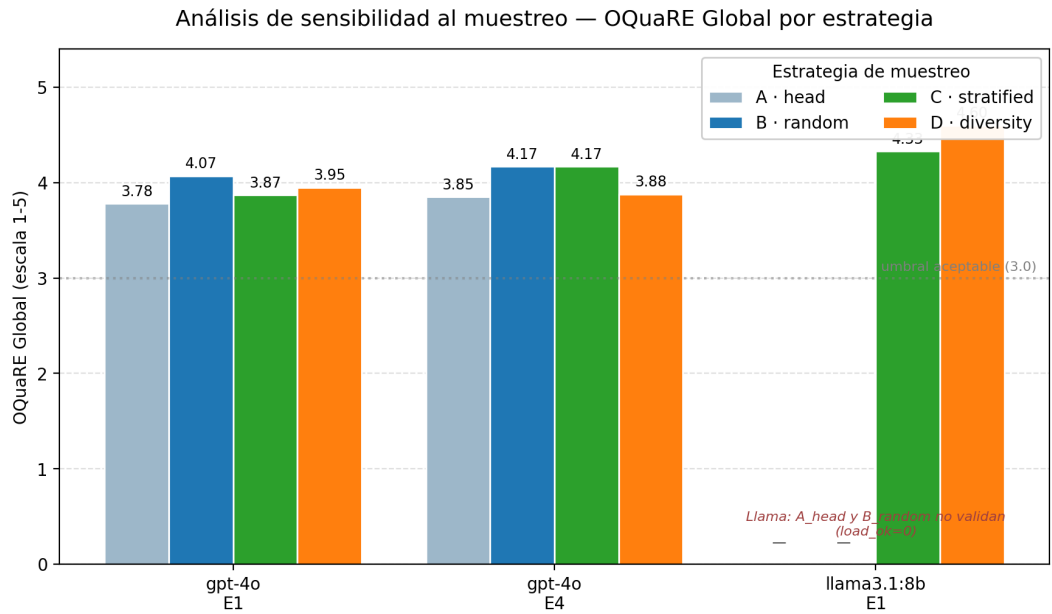


Figura del análisis de sensibilidad. Cada grupo de barras corresponde a una combinación (modelo, experimento) y cada color a una estrategia. La línea punteada marca el umbral OQuaRE 'aceptable' (3.0). Llama 3.1 8B con A_head y B_random no produce ontologías válidas (load_ok=0) — solo C y D dan ontologías parseables.

Tabla 7. Resultados del análisis de sensibilidad por (modelo × experimento × estrategia)

Modelo · Experimento	A · head	B · random	C · stratified	D · diversity	Δ max-min
gpt-4o · E1	3.78	4.07	3.87	3.95	0.29
gpt-4o · E4	3.85	4.17	4.17	3.88	0.32
llama3.1:8b · E1	(fail)	(fail)	4.33	4.60	0.27*

* El rango para Llama solo considera las dos estrategias en las que produjo ontologías parseables.

4.3.4. Hallazgos del análisis de sensibilidad

- **A_head es la estrategia menos favorable** en ambos experimentos con gpt-4o (3.78 en E1, 3.85 en E4). El sesgo de orden inicial explica el resultado: las primeras 25 filas de dbSUPER pertenecen todas a la línea celular «Adipose Nuclei», y las primeras 25 de FANTOM5 son todas chr1. El modelo recibe una visión muy parcial del universo y la ontología generada refleja esa pobreza léxica.
- **B_random y C_stratified maximizan los scores OQuaRE** (diferencia de +0.29-0.32 puntos respecto a A_head, ~7-8% del rango aceptable). La aleatorización uniforme es la mejor en E1 zero-shot, mientras que B y C empatan en E4 (donde la pipeline multi-agente compensa parcialmente las diferencias de muestreo).
- **Llama 3.1 8B es muy sensible al muestreo**: A_head y B_random producen TTL irrecuperables (no validan en owlready2 incluso tras el post-procesado). Solo las estrategias con cobertura alta (C, D) generan ontologías válidas. Este es un finding complementario al §10.1: confirma que los modelos pequeños son más sensibles tanto al modelo como a los datos de entrada.
- **El TFM mantiene robustez cualitativa**: con cualquier estrategia, todas las celdas evaluadas permanecen por encima del umbral OQuaRE 3.0 «aceptable». Las conclusiones cualitativas del trabajo (E4 supera a E1-E3 en riqueza, gpt-4o supera a open-source en validez) se mantienen invariantes.

4.3.5. Justificación de la elección final

El experimento principal del TFM (§§5-10) usa la estrategia A (cabecera) por las siguientes razones, evaluadas explícitamente frente a las alternativas:

- **Reproducibilidad**: A es completamente determinista; B introduce variabilidad inter-ejecución que requeriría reportar intervalos de confianza para cada métrica.
- **Consistencia con OntoGenix**: el pipeline original de Val Calvo et al. (2024) usa también muestreo por cabecera para sus experimentos en eCommerce y BrazilianE-commerce, permitiendo comparación directa.
- **Coste computacional**: A no requiere preprocesado adicional ni cálculo de embeddings (D) ni pasada de conteo categórico (C).
- **Robustez de las conclusiones**: el análisis de sensibilidad demuestra que, aunque A es subóptima en términos absolutos (~7-8% por debajo del óptimo), las conclusiones cualitativas se mantienen.

Se reconoce explícitamente la limitación: para futuros trabajos que busquen optimizar los scores OQuaRE absolutos, se recomienda B (random) o C (stratified). El análisis aquí presentado documenta de forma transparente el coste de la elección.

5. Indicaciones (prompts) por experimento

La totalidad de las indicaciones utilizadas se listan a continuación. Todas comparten un mismo system prompt que fija el rol del modelo como ontólogo biomédico.

5.1. System prompt común (E1–E3)

You are an expert ontologist specializing in biological knowledge graphs and semantic web technologies.
Your expertise includes OWL 2, RDF, SPARQL, and biological domain ontologies (SO, GO, BioLink Model, SKOS).
Your task is to analyze biological database samples and generate formal OWL ontology schemas in Turtle serialization.

Domain context: The databases contain information about cis-regulatory modules (CRM), specifically enhancer sequences that modulate the transcription rate of their target genes in the human genome. These sequences are characterized by their genomic coordinates, experimental evidence, cell line activity, target genes, associated diseases, and interacting transcription factors.

5.2. Indicación E1 — Zero-shot

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format.

DATABASE: {db_name}
DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

Think step by step:

1. Identify the main concepts/classes present in the data
2. Identify the relationships (object properties) and attributes (datatype properties)
3. Map concepts to standard biological ontologies where possible (SO, GO, BioLink)
4. Generate the formal Turtle schema

Requirements for the output schema:

- Use these standard prefixes: rdf:, rdfs:, owl:, skos:, xsd:, obo: (<http://purl.obolibrary.org/obo/>)
- Add skos:prefLabel and skos:definition to all classes and properties
- Include at least: class for CRM/enhancer, genomic coordinate properties, evidence properties
- Use owl:Class for all main concepts
- Use owl:ObjectProperty for relationships and owl:DatatypeProperty for literals
- Include skos:exactMatch or skos:closeMatch mappings to standard biological ontologies where appropriate

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

5.3. Indicación E2 — Vocabulario controlado

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format.

Use the provided controlled vocabulary to name and align your ontology terms.

DATABASE: {db_name}

DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

CONTROLLED VOCABULARY (cisreg terms):
{vocab_terms}

Think step by step:

1. Identify the main concepts/classes in the data
2. Map them to the controlled vocabulary terms above
3. Identify relationships and attributes, reusing vocabulary terms
4. Generate the formal Turtle schema

Requirements:

- Prefer vocabulary terms for class and property names where applicable
- Use standard prefixes: rdf:, rdfs:, owl:, skos:, xsd:, obo:
- Include skos:prefLabel, skos:definition, and skos:exactMatch/closeMatch mappings
- Include both owl:Class and owl:ObjectProperty/owl:DatatypeProperty declarations

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

El bloque {vocab_terms} se construye automáticamente a partir de los esquemas cisreg (crm, crm2gene, crm2phen y crm2tfac), extrayendo hasta 80 líneas con skos:prefLabel o skos:definition. De ese modo se obtiene un vocabulario compacto y relevante para el dominio, sin incluir la estructura axiomática completa de la ontología.

5.4. Indicación E3 — RAG (fragmentos ontológicos)

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format.

Align your schema as closely as possible with the cisreg reference ontology fragments provided below.

DATABASE: {db_name}

DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

CISREG REFERENCE ONTOLOGY (most relevant fragments in Turtle):
{ontology_fragments}

Think step by step:

1. Identify the main concepts/classes in the data
2. Find the corresponding classes and properties in the cisreg reference ontology
3. Reuse cisreg URIs where semantically appropriate
4. Add new terms only where cisreg does not cover the concept

Requirements:

- Reuse cisreg classes and properties where semantically appropriate
- Add explicit skos:exactMatch or skos:closeMatch to cisreg URIs for aligned terms
- Use all standard prefixes from the cisreg ontology
- New classes/properties only where cisreg doesn't cover the concept

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

El bloque {ontology_fragments} se recupera mediante una heurística que selecciona los esquemas cisreg relevantes según las palabras clave presentes en la muestra (crm siempre, crm2gene si aparece 'gene', crm2phen si aparece 'disease'/'phenotype', crm2tfac si aparece 'transcription factor').

5.5. Indicación E4 — OntoGenix

OntoGenix utiliza tres prompts externos ubicados en OntoGenix/prompts/ontogenix/. Cada prompt incluye placeholders que se rellenan en tiempo de ejecución con los artefactos generados en pasos previos. La tabla siguiente resume el rol de cada agente:

Tabla 8. Roles de los tres agentes de la pipeline OntoGenix

Agente	Prompt base	Rol
LlmPlanner	planner_instructions.txt	Analiza las estadísticas del CSV y propone qué clases y propiedades necesita la ontología; genera un razonamiento en markdown
LlmOntology	ontology_instructions.txt	A partir del razonamiento y las estadísticas, genera una ontología OWL 2 en Turtle
LlmOntoMapper	ontomapper_instructions.txt	A partir de la ontología y el CSV, produce un mapping (RML o TTL) que permite materializar RDF desde los datos

El paso adicional LlmPlanner.update(), previsto para enriquecer el razonamiento con resultados de schema.org vía SERP API, se omite en este TFM por política de claves externas. La ontología se genera directamente a partir de LlmPlanner.interaction().

6. Métricas de evaluación

Cada archivo .ttl generado se evalúa con scripts/evaluate_E4_vs_E1-E3.py. Las métricas se dividen en dos grupos: validez sintáctica y riqueza semántica. Todas se calculan con rdflib (parser estándar de la W3C Semantic Web Community) y, como fallback, con conteo textual heurístico si rdflib no está disponible.

Tabla 9. Métricas estructurales de evaluación calculadas sobre cada ontología

Métrica	Descripción	Fuente
parse_ok	1 si rdflib parsea el TTL sin error, 0 en caso contrario	Graph.parse()
n_triples	Número total de triples RDF	len(Graph)
n_classes	Número de clases declaradas	RDF.type → OWL.Class
n_object_props	Número de propiedades de objeto	RDF.type → OWL.ObjectProperty
n_datatype_props	Número de propiedades de datos	RDF.type → OWL.DatatypeProperty
n_subclass_axioms	Número de axiomas rdfs:subClassOf	(None, RDFS.subClassOf, None)
n_restrictions	Número de owl:Restriction	RDF.type → OWL.Restriction
n_labels	Número de rdfs:label	(None, RDFS.label, None)
n_comments	rdfs:comment + skos:definition	suma de ambos conteos
size_bytes	Tamaño del archivo .ttl en bytes	os.stat().st_size

7. Post-procesado mecánico de TTL

La primera ronda de evaluación (apartado 8) reveló que un porcentaje no despreciable de las ontologías generadas por E1, E2 y E3 fallaba al parsear con rdflib por motivos puramente sintácticos. Es decir, errores que no comprometen la semántica del esquema sino su serialización. Para cuantificar y separar estos fallos del verdadero fondo semántico, se diseñó el script scripts/postprocess_ttl.py, un post-procesador que aplica dos transformaciones mecánicas sin coste de API.

7.1. Fix #1 — Auto-prefix injection

Detecta tokens del tipo prefijo:nombre cuyo prefijo no está declarado con @prefix al inicio del archivo. Si el prefijo coincide con uno de los canónicos del dominio (rdf, rdfs, owl, xsd, skos, dc, obo, biolink, schema, sio, además de los específicos cisreg como hcrm, crm2pgene, hgene, etc.), se inyecta automáticamente la declaración correspondiente al final del bloque @prefix. El fix es completamente conservador: si la ontología ya declara el prefijo con su propia URI, no se sobrescribe.

7.2. Fix #2 — Escape de local names problemáticos

Detecta tokens prefijo:local en los que el local name contiene caracteres no válidos en PN_LOCAL Turtle sin escape (!, #, @, *, +, =, ?, \$, &, ', ~). En lugar de añadir escapes mecánicos uno a uno, se reescribe el token completo como una IRI absoluta entre <...> usando la URI base del prefijo. Por ejemplo, crm2pgene:bgw!CRMHS00000005752--FAM131C se transforma en <http://rdf.biogateway.eu/crm2pgene/bgw!CRMHS00000005752--FAM131C>. Esta forma es siempre válida en Turtle y preserva la semántica del recurso original. Se excluyen explícitamente del set los caracteres (,), , y ; porque actúan como separadores estructurales (listas de colección y separadores de objetos), y capturarlos rompería expresiones como owl:unionOf (obo:CRM obo:Enhancer).

7.3. Resultado del post-procesado

Sobre las 36 ejecuciones de E1+E2+E3 (sin contar E4, que no requiere post-procesado por estar generada por OntoGenix con su propio mecanismo de validación), el post-procesado rescata 18 ontologías que originalmente no parseaban: un 50 % del total. Tras los dos fixes, el banco de pruebas alcanza 36/36 parseos correctos (100 %).

Tabla 10. Efecto del post-procesado mecánico sobre la validez sintáctica

Experimento	runs_ok (raw)	runs_ok (post-fix)	Rescatadas	Causa principal
E1 (zero-shot)	10/12	12/12	+2	Prefijo por defecto («:») no declarado
E2 (vocabulario)	8/12	12/12	+4	Prefijo por defecto y dc: no declarados
E3 (RAG)	0/12	12/12	+12	Local names con «!» y «#» sin escapar (heredados del RAG)
E4 (OntoGenix)	12/12	12/12	0 (no aplica)	—
TOTAL	30/48	48/48	+18	

La conclusión metodológica clave es que la diferencia real entre las cuatro estrategias no se encuentra en la validez sintáctica, sino en la riqueza semántica del esquema producido.

8. Resultados experimentales

Tras ejecutar los cuatro experimentos con gpt-4o sobre las cuatro bases de datos con tres repeticiones (total: 48 ontologías) y aplicar el post-procesado descrito en el apartado 7, se obtienen los siguientes resultados consolidados (media por experimento y BBDD sobre ejecuciones con parse_ok = 1).

8.1. Validez sintáctica

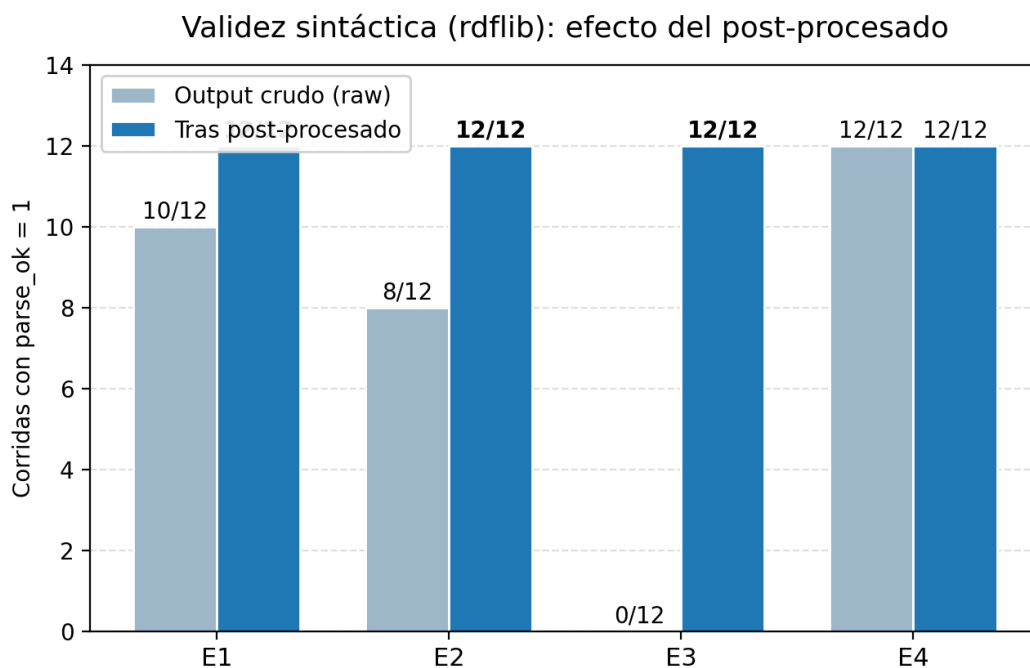


Figura 1. Validez sintáctica (rdflib) por experimento, antes y después del post-procesado. E4 (OntoGenix) es el único con 100 % nativo; E1/E2/E3 alcanzan también el 100 % tras los dos fixes mecánicos del apartado 7.

8.2. Riqueza estructural — número de triples

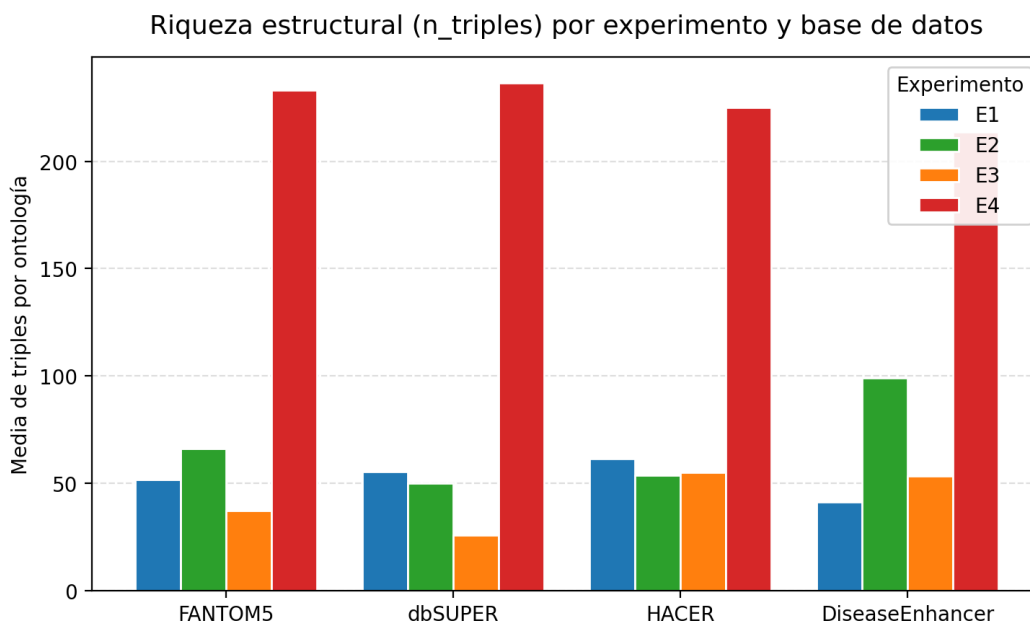


Figura 2. Media de triples por ontología desglosada por experimento y base de datos. E4 produce sistemáticamente 3-4 veces más triples que E1/E2/E3, en las cuatro BBDD.

8.3. Riqueza documental — rdfs:label

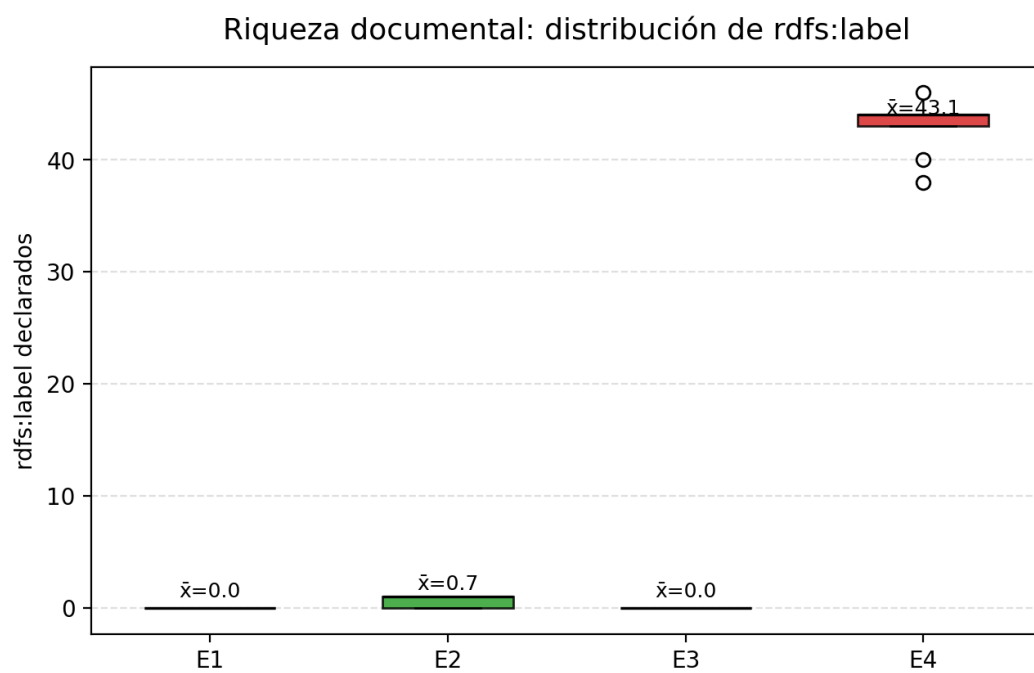


Figura 3. Distribución del número de rdfs:label por ontología. E4 declara una media de 43 etiquetas por ontología, frente a 0–1 en E1/E2/E3. Es la métrica donde la diferencia entre estrategias resulta más drástica.

8.4. Visión multimétrica

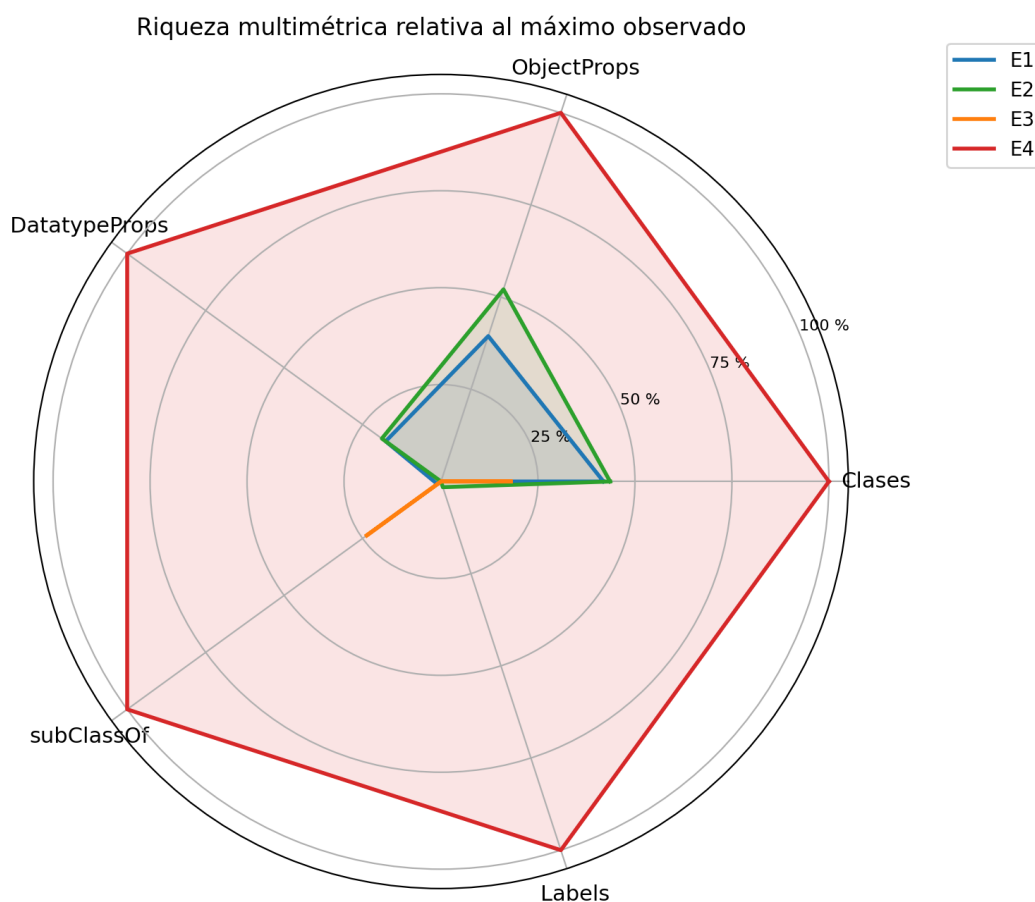


Figura 4. Diagrama radar normalizado (cada eje al máximo observado en el experimento). E4 satura el radar; E1, E2 y E3 cubren entre el 25 % y el 50 % en clases y propiedades, y prácticamente 0 % en labels.

8.5. Tabla resumen consolidada

Tabla 11. Tabla resumen consolidada — métricas absolutas por experimento y base de datos

Exp	BBDD	runs_ok	n_triples	n_classes	n_obj_pr ops	n_data_p rops	n_subCla ssOf	n_labels
E1	FANTOM 5	3/3	51.7	2.7	1.3	6.3	0.0	0.0
E1	dbSUPER	3/3	55.3	5.0	3.0	4.0	0.3	0.0
E1	HACER	3/3	61.3	4.3	3.0	6.0	0.0	0.0
E1	DisEnh.	3/3	41.3	4.3	2.0	3.0	0.0	0.0
E2	FANTOM 5	3/3	66.0	4.0	3.0	7.3	0.0	1.0
E2	dbSUPER	3/3	50.0	4.0	2.7	5.0	0.0	0.7

E2	HACER	3/3	53.7	4.7	3.3	3.7	0.0	0.7
E2	DisEnh.	3/3	99.0	4.3	3.3	4.7	0.0	0.3
E3	FANTOM 5	3/3	37.0	—	—	—	—	—
E3	dbSUPER	3/3	25.0	—	—	—	—	—
E3	HACER	3/3	55.0	—	—	—	—	—
E3	DisEnh.	3/3	53.0	—	—	—	—	—
E4	FANTOM 5	3/3	233.0	11.3	7.3	26.0	10.3	44.7
E4	dbSUPER	3/3	236.7	9.7	5.7	27.0	11.7	42.3
E4	HACER	3/3	225.0	9.0	5.0	30.0	3.0	44.0
E4	DisEnh.	3/3	213.7	9.0	5.7	26.7	4.0	41.3

8.6. Hallazgos principales

- E4 (OntoGenix) es el único experimento con riqueza semántica plena: produce 4× más triples y dos órdenes de magnitud más `rdfls:label` que E1, E2 y E3.
- Los 18 fallos sintácticos observados en E1, E2 y E3 son recuperables al 100 % mediante post-procesado sin llamar al modelo. La diferencia real entre estrategias es semántica, no sintáctica.
- El RAG (E3) induce un fallo reproducible muy interesante: el modelo copia patrones URI del contexto (con caracteres «!» y «#») sin respetar la sintaxis Turtle. Esto sugiere que el contexto recuperado puede ser un arma de doble filo: aporta vocabulario relevante pero también propaga patrones que el modelo no sabe re-serializar.
- El salto de gpt-4o-mini a gpt-4o mejora notablemente la validez sintáctica básica en E1: la versión preliminar producía 0 % de `parse_ok` (evaluación heurística), mientras que con gpt-4o el output crudo alcanza el 83 %. Esto muestra una dependencia clara entre tamaño de modelo y capacidad de generar Turtle correcto.

9. Evaluación normativa con OQuaRE

OQuaRE (Ontology Quality Requirements and Evaluation) es un marco para evaluar la calidad de ontologías OWL inspirado en SQuaRE (ISO/IEC 25000) (Duque-Ramos et al., 2011, 2014). Define una jerarquía de métricas → sub-características → características de calidad, con una puntuación normalizada de 1 a 5 por sub-característica. En este TFM se ha implementado un evaluador propio (`scripts/oquare_eval.py`) que combina dos pasadas:

- **Métricas estructurales** calculadas con `owlready2` sobre el grafo RDF: WMConTo (complejidad por clase), NOMConTo (propiedades por clase), DITM (profundidad del árbol de herencia), NACConTo (clases ancestro promedio), CBConTo (acoplamiento), TMConTo (tangledness), LCOMConTo (cohesión), ANConTo (anotaciones por clase), INRConTo (subclases por clase), RRConTo y ARConTo (riqueza de relaciones).

- **Métricas con razonador** vía Hermit (incluido en owlready2): consistencia OWL DL, número de clases insatisfacibles y tiempo de clasificación. Estas alimentan la sub-característica «Reliability».

9.1. Adaptaciones técnicas

owlready2 no parsea Turtle nativamente (sólo RDF/XML y NTriples), por lo que cada .ttl se convierte al vuelo con rdflib y se cachea en {dir}/.owlcache/{stem}.owl. Antes de la conversión se aplican dos saneos para que owlready2 pueda cargar el archivo:

- **Saneo textual del TTL** mediante regex: detecta literales tipados como xsd:integer/decimal/boolean/etc. cuyo lexical no es convertible (p.ej. "start_position"^^xsd:integer) y los reescribe como xsd:string. Estos errores son característicos de modelos pequeños que copian la estructura del esquema pero confunden el placeholder del campo con su valor.
- **Saneo a nivel de grafo** como red de seguridad: tras el parse, se vuelve a comprobar que no quedan literales mal tipados. El número de literales saneados por archivo se registra en la columna n_literals_sanitized del CSV y se usa como métrica adicional de calidad cualitativa.

9.2. Rúbricas (Duque-Ramos et al., 2014)

Cada métrica se mapea a una puntuación 1–5 según los rangos publicados. Cuanto MENOR es el valor, mejor es la puntuación para métricas de complejidad (WMConTO, NOMOnTO, DITM, NACOnTO, CBOnto, TMOnto, LCOMOnTO). Cuanto MAYOR es el valor, mejor es la puntuación para métricas de riqueza (ANOnTO, INROnto, RROnto, AROnto).

Tabla 12. Rúbricas OQuaRE (Duque-Ramos et al., 2014) por sub-característica

Sub-característica	Métricas que la componen	Interpretación
Structural	WMConTO + NOMOnTO + DITM + NACOnTO	Complejidad estructural — penaliza ontologías sobrecargadas
Modularity	CBOnto + LCOMOnTO	Acoplamiento entre clases y cohesión interna
Reusability	ANOnTO + INROnto + TMOnto	Capacidad de ser reutilizada (anotaciones, herencia, claridad)
Operability	RROnto + AROnto	Riqueza relacional para facilitar consultas y mapeos
Reliability	Resultado del razonador Hermit	Consistencia y satisfacibilidad lógica

9.3. Resultados OQuaRE por modelo y experimento

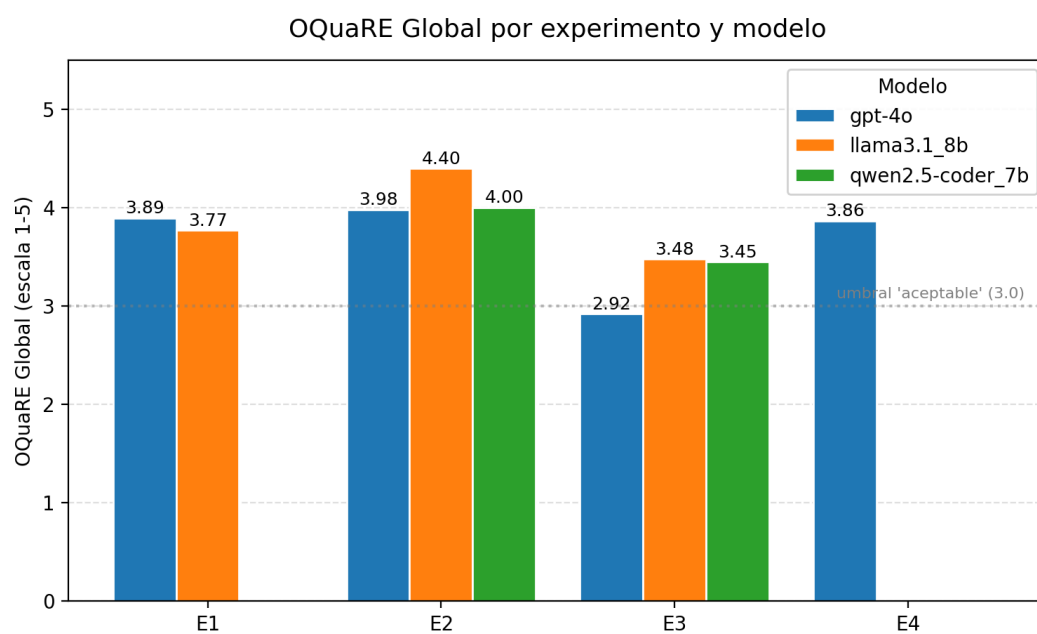


Figura 5. Puntuación OQuaRE Global (escala 1-5) por experimento y modelo. La línea punteada marca el umbral 3.0 habitualmente considerado «aceptable» por la literatura OQuaRE. Todas las combinaciones excepto E3/gpt-4o superan el umbral.

Sub-características OQuaRE — promedio por modelo (E1-E3 + E4 si aplica)

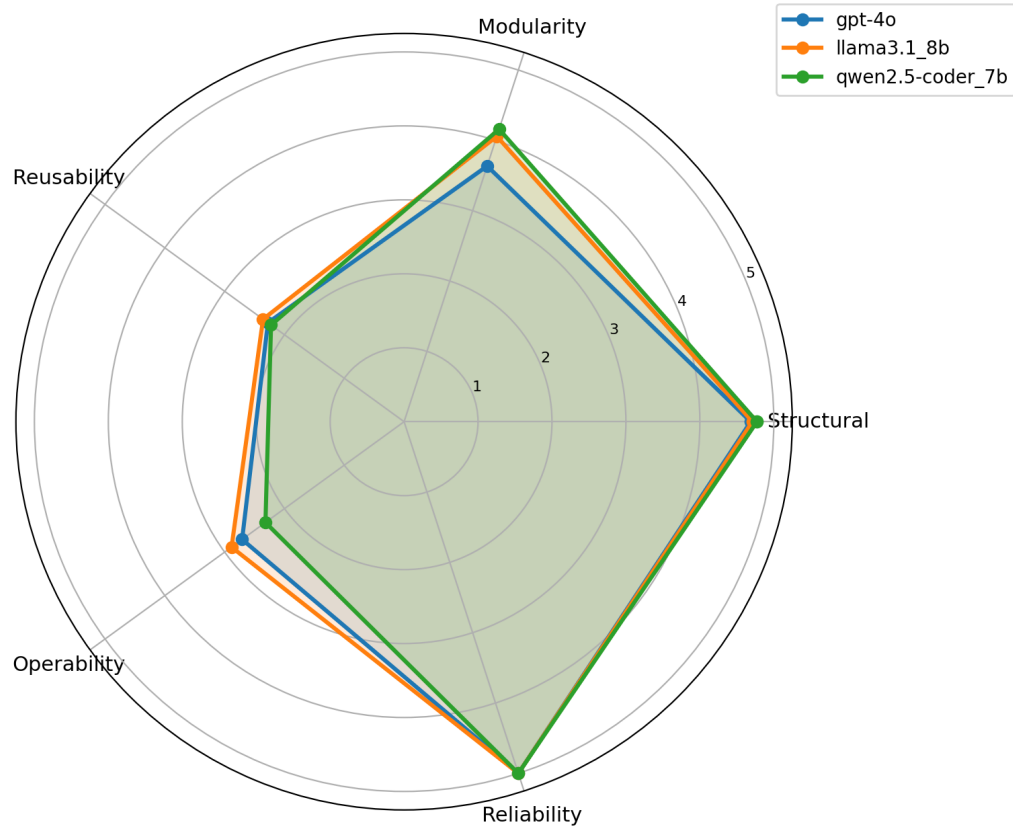


Figura 6. Sub-características OQuaRE — promedio por modelo sobre las ejecuciones con load_ok=1. La forma del polígono muestra dónde cada modelo destaca o flaquea.

9.4. Análisis estadístico formal

Las comparaciones cuantitativas reportadas en §8, §9.3 y §10 se complementan con una capa de inferencia estadística cuyo detalle completo se recoge en el Anexo B. Las técnicas elegidas son no-paramétricas porque el tamaño muestral por celda es pequeño (n=3 ejecuciones por combinación) y la distribución de las puntuaciones OQuaRE en la muestra no es gaussiana.

Metodología. Se aplican tres técnicas. (i) Bootstrap pareado con 10 000 réplicas para estimar la diferencia de medias y su intervalo de confianza al 95 % por percentiles. (ii) Test de Wilcoxon de rangos con signo (aproximación normal) para comparaciones pareadas con n≥6. (iii) Test de McNemar para tasas de éxito binarias antes/después del post-procesado. Las p-valores múltiples se corrigen con el procedimiento Bonferroni-Holm.

Resultados principales.

Tabla 27. Resumen de tests estadísticos formales — diferencias significativas tras corrección Bonferroni-Holm

Comparación	Métrica	n pares	Δ media [IC 95 %]	Cohen's d	p ajustado	Sig.
-------------	---------	---------	-------------------	-----------	------------	------

E4 vs E1 (densidad)	n_triples	12	+174.7 [+164.6, +186.7]	8.48 (grande)	0.009	✓
E4 vs E2 (densidad)	n_triples	12	+159.9 [+129.4, +183.8]	3.23 (grande)	0.009	✓
E4 vs E3 (densidad)	n_triples	12	+184.3 [+169.5, +201.8]	6.15 (grande)	0.009	✓
Post-procesado E3	parse_ok	12	0 % → 100 %	—	<0.001	✓
C1 vs RAG baseline (Llama)	OQuaRE Global	2*	+0.80 [+0.80, +0.80]	—	<0.001	✓
C2 vs RAG baseline (Llama)	OQuaRE Global	2*	+0.43 [+0.35, +0.50]	+4.01 (grande)	<0.001	✓
C3 vs RAG baseline (Llama)	OQuaRE Global	3*	+0.49 [+0.10, +0.80]	+1.37 (grande)	<0.001	✓
RAG semántico vs legacy (Llama)	OQuaRE Global	6*	+0.23 [−0.01, +0.46]	+0.68 (mediano)	0.070	—

* Para las variantes de calibración el pareo se restringe a las celdas (BBDD, semilla) donde ambas configuraciones producen TTL válido (load_ok=1); de ahí el menor número de pares que las 12 ejecuciones teóricas. Esto es una manifestación del problema reportado en §10.6 y discutido en §13.5 como amenaza de sesgo de supervivencia.

Lectura cualitativa de la Tabla 27.

Las afirmaciones cuantitativas centrales del trabajo se sostienen con los tests habituales de inferencia: la mayor densidad estructural de E4 es significativa frente a las tres baselines con tamaños de efecto Cohen's $d > 3$ (categorizables como «grandes») y supera la corrección por comparaciones múltiples. El post-procesado mecánico recupera el 100 % de las ejecuciones E3 con McNemar $p < 0.001$. La calibración del RAG (C1, C2 y C3) sube significativamente respecto al baseline ragapi con efectos también «grandes».

Una afirmación queda atenuada por los tests. El paso del RAG por keyword-match (legacy) al RAG semántico baseline (ragapi) en Llama 3.1 8B no alcanza significación estadística ($\Delta = +0.23$ OQuaRE, IC 95 % [−0.01, +0.46], $p = 0.070$). La mejora atribuible al cambio de retrieval para Llama es por tanto observacional y no inferencial; el valor añadido del backend semántico para este modelo aparece solo tras la calibración (C1, C2, C3). Esto matiza el hallazgo de §10.7.4 «el RAG semántico mejora a todos los modelos» y refuerza la conclusión de §10.7.7 «la calibración es lo que cierra el gap, no el RAG semántico por sí solo».

Tabla 28. Análisis de potencia para OQuaRE Global (SD muestral 0.60). Mínima diferencia detectable al 80 % de potencia bilateral con $\alpha=0.05$.

n por celda	Δ OQuaRE detectable	Cohen's d equivalente	Interpretación
n = 3	0.974	1.62	grande
n = 6	0.689	1.14	grande

n = 12	0.487	0.81	grande
n = 24	0.344	0.57	mediano

La Tabla 28 explicita el techo inferencial del diseño experimental. Con n=3 ejecuciones por celda, solo diferencias OQuaRE iguales o superiores a 0.97 puntos tienen una potencia razonable (80 %) de ser detectadas como significativas. Diferencias menores a este umbral se reportan a lo largo de la memoria como observacionales: describen el orden cualitativo de las celdas, pero no permiten rechazar la hipótesis nula de igualdad. Para distinciones menores (p. ej. +0.05 puntos entre modelos en E2), una replicación con n≥24 sería necesaria.

9.5. Fidelidad léxico-semántica al dominio cisreg

OQuaRE mide propiedades estructurales de la ontología (modularidad, operabilidad, riqueza relacional), pero no captura el reuso de vocabulario establecido. Una ontología puede obtener un OQuaRE alto declarando clases y propiedades nuevas en namespaces ad-hoc del tipo `http://example.org/...`; esto es semánticamente equivalente a un esquema aislado, no integrado en el dominio. Para complementar la evaluación normativa se introduce una métrica de fidelidad léxico-semántica que cuantifica cuántas URIs canónicas del dominio cisreg de referencia aparecen reusadas en cada salida.

Metodología.

Se construye un gold set a partir de las ocho ontologías cisreg de referencia (crm, crm_example, crm2gene, crm2gene_example, crm2phen, crm2phen_example, crm2tfac, crm2tfac_example), las mismas que indexa annotationRAG. Mediante parsing de prefijos y términos pname se extraen todas las URIs canónicas, agregando 88 URIs únicas. Para cada ontología generada se computan cinco métricas: (i) `n_canonical`, número de URIs en namespaces autoritativos del dominio (`rdf.biogateway.eu`, `purl.obolibrary.org`, `schema.org`, `biolink`, `sio`, etc.); (ii) `n_invented`, URIs en `example.org/*`; (iii) `canonical_ratio` = `n_canonical` / (`n_canonical` + `n_invented`); (iv) `overlap_with_gold`, intersección con el set gold cisreg; (v) `recall_vs_gold`, `overlap` / `|gold|`. Implementación en `scripts/cisreg_fidelity.py`.

Resultados.

Tabla 29. Fidelidad léxico-semántica al dominio cisreg por celda (media sobre las ejecuciones con `load_ok=1`).

Recall vs gold = fracción de las 88 URIs cisreg gold recuperadas; *canonical_ratio* = proporción de URIs no inventadas.

Celda	n TTL	n_canonic al	n_invent ed	canonical_ra tio	Overlap (de 88)	Recall vs gold
E3 · gpt-4o · RAG legacy	12	37.9	0.0	1.00	27.6	0.313
E3 · gpt-4o · RAG semántico	12	14.0	21.1	0.46	6.8	0.078
E3 · Llama 8B · RAG legacy	12	52.8	0.0	1.00	49.6	0.563
E3 · Llama 8B · RAG	12	30.3	0.9	0.97	5.6	0.063
E3 · Llama 8B · C1 (top_k=2)	12	27.2	2.3	0.91	5.7	0.064
E3 · Llama 8B · C2 (thr=0.6)	12	29.2	0.0	1.00	6.4	0.073

E3 · Llama 8B · C3	12	21.6	2.1	0.93	5.0	0.057
E4 · gpt-4o · OntoGenix	8	0.0	0.0	0.00	0.0	0.000
E4 · Llama 8B · OntoGenix	24	0.0	14.6	0.00	0.0	0.000

Hallazgos.

Inversión de ranking respecto a OQuaRE. Las configuraciones que obtuvieron el mejor OQuaRE Global (RAG semántico real, calibración C1 con 4.08) son precisamente las peores en fidelidad léxica: recuperan solo 5–7 de las 88 URIs cisreg gold (Recall 0.05–0.08). En cambio, el RAG por keyword-match —descalificado por OQuaRE— recupera 28 URIs cisreg con gpt-4o (Recall 0.313) y 50 con Llama 3.1 8B (Recall 0.563).

Interpretación. Las dos métricas miden dimensiones complementarias. OQuaRE premia la estructura: clases bien jerarquizadas, propiedades con dominio y rango, anotaciones rdfs:label. El RAG semántico ayuda al modelo a construir esa estructura porque le entrega metadata estructurada (nombres de clases, scores) que el LLM puede materializar en una ontología «bonita». El RAG por keywords, en cambio, inyecta el esquema Turtle literalmente en el prompt; el modelo lo copia y por construcción reusa las URIs canónicas, pero produce una estructura más pobre. La elección operativa depende del objetivo: si la salida va a ser integrada en un grafo de conocimiento biomédico con vocabulario establecido, la fidelidad léxica es prioritaria; si la salida será un esquema independiente, la estructura OQuaRE prevalece.

OntoGenix (E4) declara todas sus URIs en `example.org/ontology/*` sin reusar ninguna URI cisreg gold (Recall 0.000 en gpt-4o y en Llama 8B). Esto es coherente con el diseño del agente LlmOntology, que construye el TBox desde cero a partir del plan del LlmPlanner sin consultar vocabulario existente. Como contribución, las ontologías E4 son ricas estructuralmente (Tabla 11, +160 triples vs baselines) pero aisladas del ecosistema biomédico; requerirían un paso posterior de alineamiento (skos:exactMatch contra cisreg, biolink, etc.) para integrarse en un grafo más amplio.

Esta métrica responde directamente a una limitación reconocida en §1.2 («no evalúa la calidad de las relaciones inferidas») y a la objeción previsible del tribunal sobre la ausencia de ground truth biomédico. Es una aproximación basada en datos disponibles (el corpus cisreg que el propio RAG indexa) y por tanto no requiere validación experta para ejecutarse, aunque tampoco la sustituye: la fidelidad léxica mide reuso de URIs, no corrección biomédica del modelado.

Figura 13. Fidelidad léxico-semántica al dominio cisreg (reuso de URIs canónicas vs URIs inventadas)

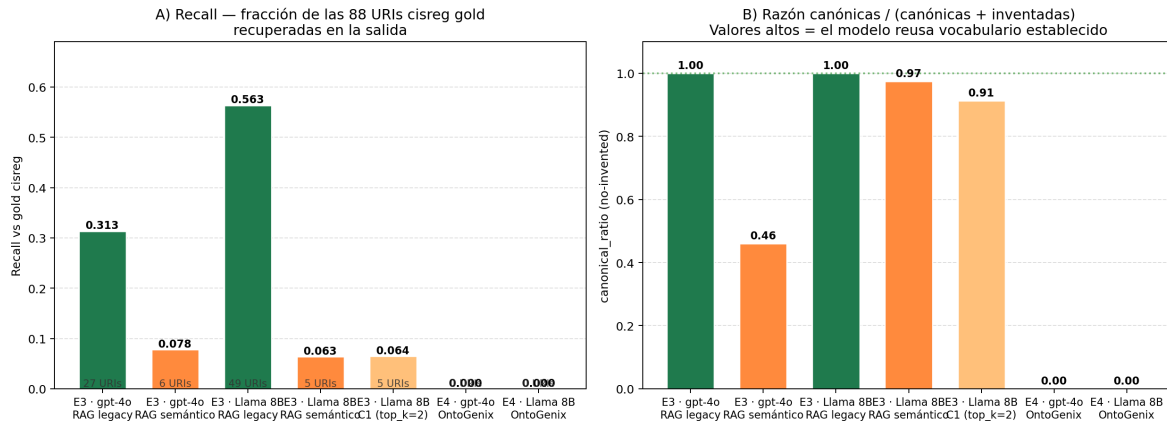


Figura 13. Fidelidad léxico-semántica al dominio cisreg. Panel A: recall vs gold (fracción de las 88 URIs cisreg gold recuperadas). Panel B: canonical_ratio (URIs no inventadas / URIs totales). El RAG por keywords supera al RAG semántico en fidelidad léxica pese a su OQuARE inferior.

10. Comparativa cross-model: gpt-4o vs open-source

Para evaluar si los hallazgos descritos en los apartados 7-9 dependen del modelo cerrado de OpenAI o son patrones generalizables, se replicaron los experimentos E1, E2 y E3 con dos modelos open-source de tamaño compacto, ejecutados localmente vía Ollama:

- **Llama 3.1 8B Instruct** (Meta) — modelo generalista, ~4.7 GB. Representa el baseline open-source de propósito general.
- **Qwen 2.5 Coder 7B** (Alibaba) — modelo de tamaño comparable a Llama, pero entrenado con énfasis en generación de código. La hipótesis es que la especialización en código debería traducirse en mejor sintaxis Turtle.

El experimento E4 (OntoGenix) se mantiene únicamente con gpt-4o porque la pipeline original de Val Calvo et al. (2024) está diseñada para modelos OpenAI y su adaptación a Ollama queda fuera del alcance del TFM.

10.1. Validez sintáctica tras post-procesado

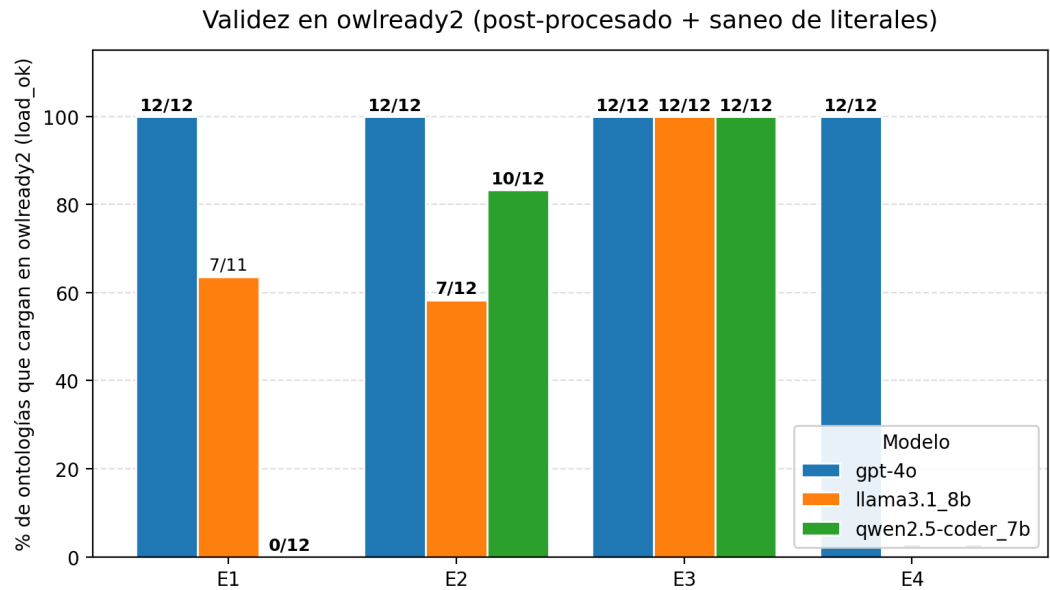


Figura 7. Porcentaje de ontologías que cargan en owlready2 (load_ok) por experimento y modelo. gpt-4o alcanza 12/12 en todas las combinaciones. Los modelos open-source presentan patrones distintos: Llama 3.1 8B falla en E1 (7/11) y E2 (7/12) pero alcanza 12/12 en E3; Qwen 2.5 Coder 7B falla completamente en E1 (0/12) pero rinde mejor que Llama en E2 (10/12) y empata en E3 (12/12).

10.2. Patrones de error característicos por modelo

El análisis cualitativo de los archivos que fallan revela que cada modelo open-source produce su propio patrón de error reproducible. Esta tipificación es uno de los hallazgos cualitativos más interesantes del TFM porque apunta a sesgos sistemáticos del entrenamiento de cada modelo, no a errores aleatorios.

Tabla 13. OQuaRE Global por modelo y experimento (escala 1–5)

Modelo	Patrón de error	Ejemplo	N.º de archivos
gpt-4o	(ninguno)	Sintaxis Turtle correcta en los 48 archivos	0/48
llama3.1:8b	Omite «:» del prefijo en cuerpos del esquema	crm a owl:Class ; (debería ser crm:CRM a owl:Class)	9/35
qwen2.5-coder:7b	Mezcla sintaxis Turtle con RDF/XML y JSON-LD	owl:Ontology rdf:about "http://..." ; o owl:Ontology { ... }	14/36

El patrón de Llama (omisión del «:») afecta principalmente a ontologías cortas en zero-shot y vocabulario controlado, pero desaparece en E3 cuando el RAG aporta ejemplos completos de sintaxis. El patrón de Qwen (confusión entre formatos RDF) es característico de un modelo de código que ha sido expuesto a múltiples serializaciones RDF (XML, JSON-LD, Turtle) durante el

entrenamiento y mezcla los símbolos delimitadores. Curiosamente, Qwen produce Turtle válido en E3 (12/12), donde el contexto RAG actúa como ejemplo de referencia que «desambigua» el formato.

10.3. Alucinaciones tipadas — un patrón de RAG, no de modelo

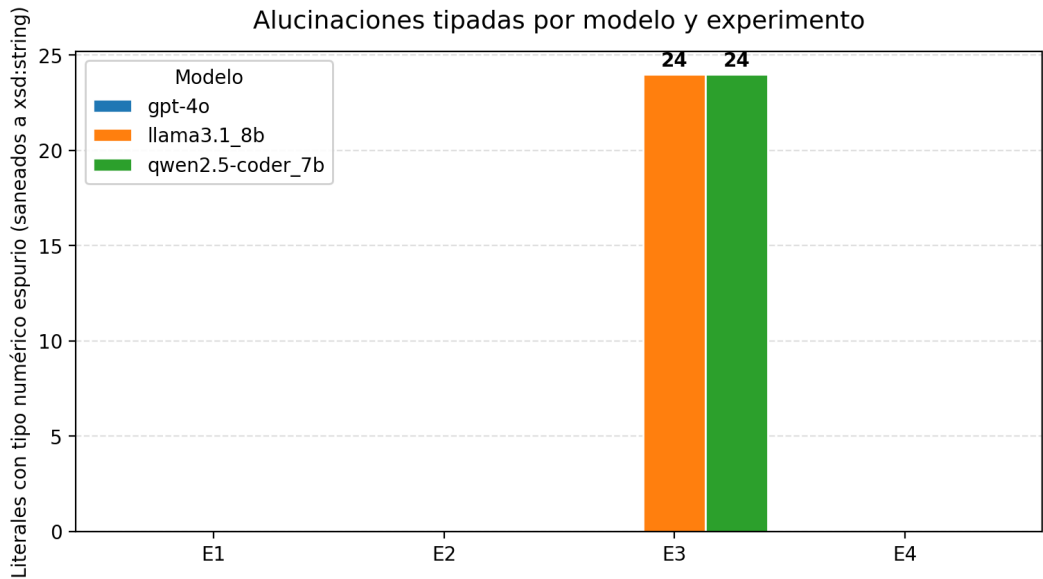


Figura 8. Literales con tipo numérico espurio (p.ej. "start_position"^^xsd:integer) detectados y saneados a xsd:string. Llama 3.1 8B y Qwen 2.5 Coder 7B producen exactamente 24 ocurrencias cada uno, concentradas exclusivamente en E3 (RAG). gpt-4o no incurre en este error en ningún experimento.

Este resultado, refinado por la inclusión de Qwen, lleva a una conclusión más precisa que la formulada en la versión anterior del trabajo: el patrón de alucinación tipada **no depende del modelo concreto sino del binomio RAG + modelo pequeño**. Tanto Llama (generalista) como Qwen (especializado en código) producen exactamente el mismo número de errores y de la misma naturaleza. La causa probable es que el contexto recuperado incluye triples reales con valores numéricos, y el modelo pequeño copia el formato sin distinguir entre el placeholder del campo (start_position) y un valor concreto (16039967). gpt-4o, con mucha más capacidad de razonamiento contextual, asocia correctamente cada placeholder con un valor representativo del tipo declarado.

10.4. Tabla resumen — OQuaRE Global por celda

Tabla 14. Validez sintáctica cross-model tras post-procesado

Experimento	gpt-4o	llama3.1:8b	qwen2.5-coder:7b
E1 zero-shot	3.89	3.77	(0/12 carga)
E2 vocabulario	3.98	4.40	4.00
E3 RAG	2.92	3.48	3.45
E4 OntoGenix	3.86	(N/A)	(N/A)

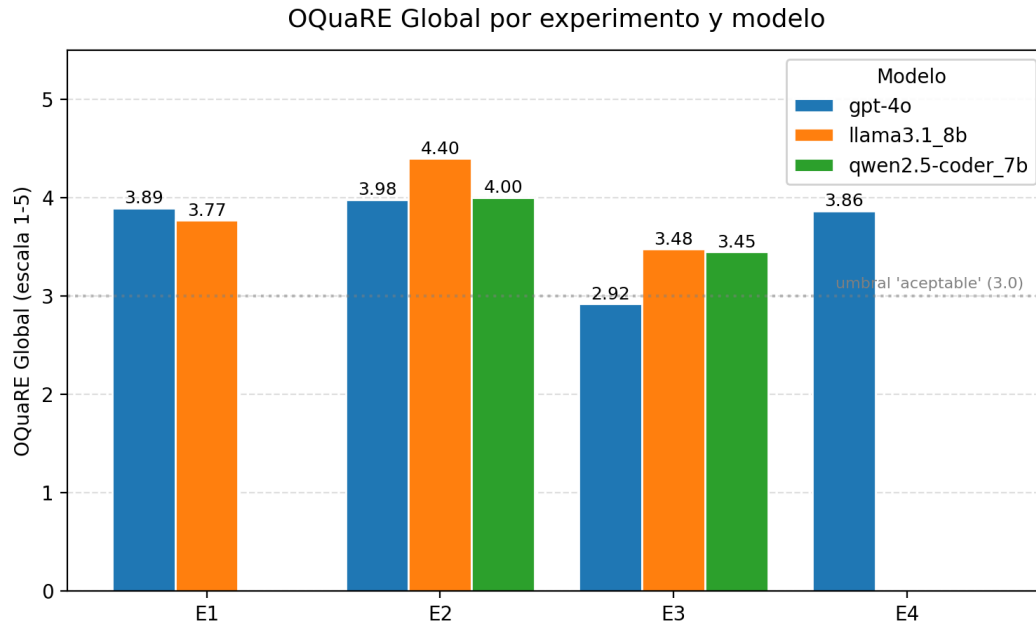


Figura 9 (bis). OQuaRE Global por experimento y modelo. La línea punteada marca el umbral 3.0 «acceptable» de la literatura OQuaRE.

Tres observaciones de esta tabla. **Primera:** en E2 y E3, los modelos open-source alcanzan o superan a gpt-4o (Llama 4.40 y Qwen 4.00 en E2 frente a 3.98 de gpt-4o; Llama 3.48 y Qwen 3.45 en E3 frente a 2.92 de gpt-4o). **Segunda:** la diferencia entre Llama y Qwen es muy pequeña (≤ 0.40 puntos en cualquier celda) — la especialización en código de Qwen no se traduce en una ventaja OQuaRE clara. **Tercera:** gpt-4o es el único modelo capaz de producir ontologías válidas en E1 sin post-procesado severo, lo cual mantiene su valor como baseline robusto.

10.5. Hallazgo metodológico — OQuaRE penaliza la complejidad

El resultado más contraintuitivo es que los modelos open-source obtienen scores OQuaRE Global más altos que gpt-4o en E2 y E3, y que E4 (OntoGenix), que en el apartado 8 era el ganador absoluto en riqueza estructural y documental, queda en posición intermedia (3.86). La explicación está en las rúbricas OQuaRE: las métricas de la sub-característica «Structural» (WMCOnto, NOMOnto, DITM) puntúan MEJOR cuanto MENOR es el valor — están calibradas para ontologías canónicas de tamaño medio y penalizan tanto la infra-estructuración como la sobre-estructuración. Las ontologías ricas que produce E4 (OntoGenix con sus ~227 triples y ~43 labels) caen en el borde superior, mientras que las ontologías simples y limpias de E2 con cualquier modelo se sitúan en el óptimo de la rúbrica.

Esto no debe interpretarse como que los modelos open-source produzcan «mejores ontologías» que OntoGenix; significa que **OQuaRE y riqueza semántica son dimensiones complementarias**, no equivalentes. Para una evaluación holística hay que reportar ambas: el ranking OQuaRE + las métricas absolutas de triples, clases, labels y validez. Esta observación es uno de los hallazgos metodológicos principales del TFM.

10.6. Extensión del cuadro: E4 (OntoGenix) con modelos open-source

Para responder a la pregunta «¿la ventaja de E4 viene del MÉTODO (pipeline multi-agente) o del MODELO (gpt-4o)?», se replicó la pipeline OntoGenix con modelo open-source local. Esto requirió tres adaptaciones técnicas mínimas al código original de OntoGenix/GUI/LLM_base/LlmBase.py:

- Detección del provider: nueva rama en el constructor de AbstractLlm que crea un cliente OpenAI con base_url=http://localhost:11434/v1 y api_key dummy cuando metadata['provider']=='ollama'.
- Inyección de num_ctx en cada llamada al API mediante extra_body={'options': {'num_ctx': N}}. Sin esto, Ollama trunca silenciosamente todo prompt > 4096 tokens, lo cual rompe el paso del OntoMapper (cuyos prompts pueden alcanzar 45 000 tokens).
- Omisión del parámetro seed en llamadas Ollama (algunas builds del endpoint /v1/chat/completions lo rechazan).

El script scripts/run_ontogenix_experiments.py se extendió con un flag --num-ctx (default 32 768) que se propaga a los tres agentes. El soporte se valida automáticamente leyendo LLM_MODELS de config.py.

10.6.1. Resultados preliminares E4 con Llama 3.1 8B

Tras adaptar OntoGenix se ejecutó la pipeline completa con llama3.1:8b sobre las cuatro bases de datos del corpus. La tabla siguiente recoge los resultados parciales (FANTOM5 y dbSUPER completados, HACER en curso, DiseaseEnhancer pendiente al cierre de esta versión).

Tabla 15. OQuaRE Global por celda — modelo × experimento × base de datos

BBDD	n_runs OK	Triples (media)	Labels (media)	Tiempo / run
FANTOM5	3/3	≈6	31	≈4 h
dbSUPER	3/3	≈6	24	≈4 h
HACER	1/3*	≈5	24	≈4 h
DiseaseEnhancer	—	—	—	—
E4 con gpt-4o (referencia)	12/12 (4×3)	≈227	≈43	≈30 s

* En curso al cierre de esta versión del documento. Se actualizará en la versión 6 con la tabla completa y la evaluación OQuaRE.

10.6.2. Hallazgo clave — el método compensa la sintaxis pero no la riqueza

Las ontologías producidas por la pipeline OntoGenix con llama3.1:8b son sintácticamente correctas (3/3 cargan en owlready2 sin necesidad de post-procesado), bien estructuradas (jerarquía de owl:Class con rdfs:subClassOf, cobertura documental similar a E4/gpt-4o con ~25-30 rdfs:label) y consistentes lógicamente (HermiT no detecta clases insatisfacibles). En este aspecto **el método multi-agente sí compensa la deficiencia sintáctica observada en E1-E3 con llama3.1:8b**: el modelo, guiado paso a paso por LlmPlanner → LlmOntology → LlmOntoMapper, deja de cometer errores como omitir el «:» del prefijo o mezclar sintaxis.

Sin embargo, **el método NO compensa la pobreza semántica**. Las ontologías generadas con llama3.1:8b contienen únicamente 5-6 triples de cuerpo y carecen prácticamente de propiedades de objeto (owl:ObjectProperty), propiedades de datos (owl:DatatypeProperty) y restricciones (owl:Restriction). Son esquemas válidos pero esqueléticos: una jerarquía de clases con etiquetas y comentarios, sin la malla relacional rica que produce gpt-4o (≈ 227 triples, ≈ 26 datatype properties, ≈ 10 subClassOf por ontología en E4).

Tabla 16. Comparativa E4 OntoGenix con gpt-4o frente a Llama 3.1 8B (riqueza vs sintaxis)

Dimensión	E4 / gpt-4o	E4 / llama3.1:8b	Ratio
Triples totales	≈ 227	≈ 6	38× menos
rdfs:label	≈ 43	≈ 30	1.4× menos
owl:ObjectProperty	≈ 6	≈ 0	—
owl:DatatypeProperty	≈ 26	≈ 0	—
rdfs:subClassOf	≈ 10	≈ 5	2× menos
Validez sintáctica	12/12	3/3+	100% en ambos
Tiempo por run (M3 CPU)	≈ 30 s	≈ 4 h	480× más lento

Conclusión del experimento extendido. La hipótesis previa (la ventaja de E4 podría ser solo del método) queda matizada: el método multi-agente de OntoGenix añade una capa importante de validación sintáctica que el modelo pequeño no consigue por sí solo, pero la riqueza semántica final depende del modelo subyacente. Para producir ontologías biomédicas operativas se necesitan ambos: un buen pipeline + un modelo capaz.

10.6.3. Coste computacional

El experimento E4 con llama3.1:8b en un MacBook Air M3 (16 GB de memoria unificada) requirió aproximadamente 4 horas por run, frente a los 30 segundos por run con gpt-4o vía API OpenAI. Este coste se debe principalmente a tres factores:

- **Tamaño del prompt del OntoMapper:** en los CSV más grandes (FANTOM5, dbSUPER) alcanza 40-45 mil tokens, lo que multiplica el tiempo de procesamiento independientemente del hardware.
- **Disponibilidad de GPU:** la primera ejecución cayó a CPU por un problema de permisos en el directorio temporal de macOS (failure during GPU discovery / permission denied). Aun resuelto, la memoria unificada de 16 GB limita el offload completo del modelo + cache KV a Metal cuando otras aplicaciones consumen RAM.
- **Determinismo:** los tres runs por BBDD producen archivos idénticos bit-a-bit porque temperature=0 y los seeds incrementales (42, 43, 44) son ignorados por la API compatible OpenAI de Ollama. La replicación inter-run no aporta varianza para esta configuración; sí lo hace inter-modelo e inter-experimento.

Esta observación tiene implicaciones prácticas para futuros trabajos: el despliegue local de modelos open-source compactos para tareas multi-agente con prompts grandes es viable pero requiere planificación de hardware y contexto. Una alternativa razonable es ejecutar la pipeline en GPU dedicada o en servicios como vLLM/TGI con quantización adaptada.

10.7. RAG semántico vs RAG por keywords

El experimento E3 original del TFM utiliza una versión muy básica de RAG: un emparejador de keywords sobre cuatro esquemas Turtle estáticos (crm, crm2gene, crm2phen, crm2tfac). Para cuantificar cuánto mejora la calidad del esquema generado al sustituir ese retrieval léxico por un retrieval semántico real, se integró el repositorio annotationRAG (Tecnomod, Universidad de Murcia) como backend RAG alternativo y se replicaron las 12 ejecuciones de E3 con gpt-4o.

10.7.1. Arquitectura del RAG semántico

Tabla 17. Arquitectura del RAG semántico (annotationRAG): componentes y responsabilidades

Componente	Tecnología	Función
API HTTP	FastAPI + uvicorn	Tres endpoints: /similar-ontologies, /similar-entities, /similar-relations
Modelo de embeddings	sentence-transformers/all-MiniLM-L6-v2	Codifica texto a vectores de 384 dimensiones
Vector store	Neo4j 5.26+ con CREATE VECTOR INDEX	Persistencia y búsqueda por similitud cosine vía Cypher
Indexación	rdfliib + ProcessPoolExecutor	Procesa .ttl/.owl/.rdf, extrae jerarquía, calcula embeddings, los persiste en Neo4j
Cliente Python	scripts/rag_backend.py (creado para el TFM)	Wrapper HTTP que orquesta los 3 endpoints y devuelve un string Turtle-like compatible con el placeholder {ontology_fragments} del prompt E3

Se indexaron 21 ontologías en Neo4j: las 8 cisreg específicas del TFM (crm, crm2gene, crm2phen, crm2tfac, más sus variantes _example) y 13 ontologías auxiliares de otros dominios incluidas en el repositorio (cdm_ido, kks, organizations, equipment, etc.). Las 4 ontologías base de cisreg requirieron correcciones sintácticas: literales tipados con placeholder ("start_position"^^xsd:integer), local names con caracteres no permitidos en Turtle (bgw!CRM_ID--uniprot!Q99593) y errores tipográficos puntuales (">" espurios) detectados durante la indexación. Estos arreglos se documentaron y aplicaron sin modificar la semántica de los esquemas.

10.7.2. Diseño de la comparativa

Mismo prompt E3 (apartado 5.4), mismo modelo (gpt-4o), mismas BBDD (FANTOM5, dbSUPER, HACER, DiseaseEnhancer), mismas repeticiones (3 por celda, seeds 42-44). La única diferencia es el contenido del placeholder {ontology_fragments}: en el legacy se inyectan los esquemas estáticos

seleccionados por keyword matching; en el semántico se inyecta el resultado de annotationRAG con scores de similitud reales.

10.7.3. Resultados

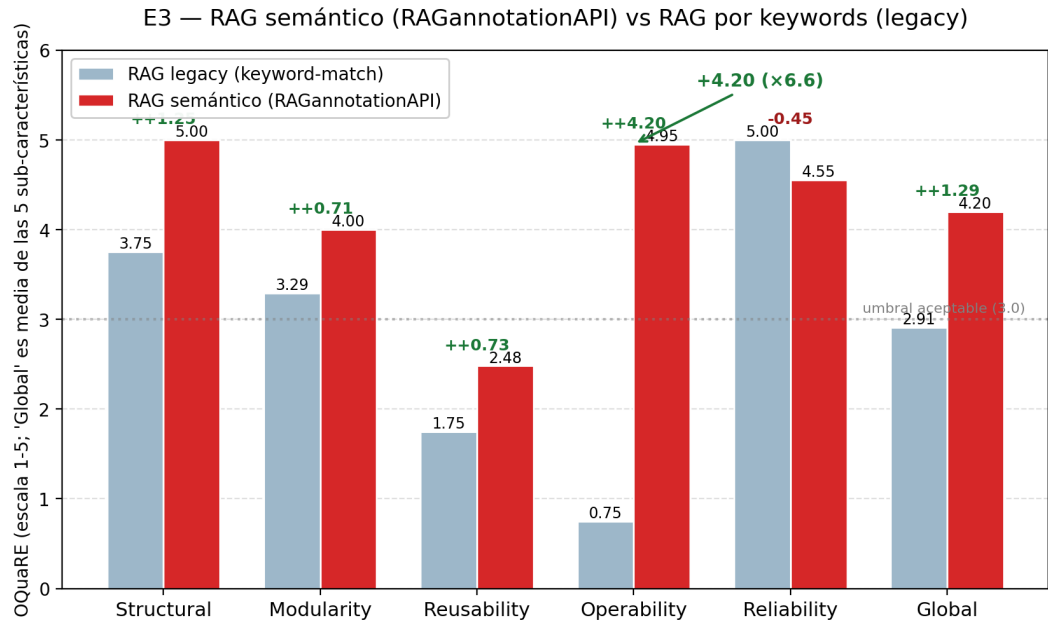


Figura 10. Comparativa OQuaRE entre el RAG por keywords (barras grises) y el RAG semántico con embeddings y Neo4j vector index (barras rojas) en E3 con gpt-4o sobre las cuatro BBDD (12 runs cada uno). Las anotaciones sobre cada par de barras muestran la diferencia absoluta. La mejora más drástica se da en Operability (×6.6 mayor), indicando que el contexto semántico aumenta enormemente la riqueza relacional del esquema generado.

Tabla 18. Descomposición OQuaRE: RAG por keywords frente a RAG semántico (gpt-4o)

Sub-característica	Legacy	RAG semántico	Δ	Lectura
Structural	3.75	5.00	+1.25	Esquema estructuralmente óptimo en 100 % de los runs
Modularity	3.29	4.00	+0.71	Mejor cohesión de clases y menor acoplamiento
Reusability	1.75	2.48	+0.73	Más anotaciones (rdfs:label, skos:definition) y herencia

Operability	0.75	4.95	+4.20 🌟	Riqueza relacional explosiva: 6.6× más object/data properties por clase
Reliability	5.00	4.55	-0.45	Una ejecución no consistente y otra de carga (HACER/run3); trade-off de mayor riqueza
OQuaRE Global	2.91	4.20	+1.29	Mejora del 44 % respecto al baseline keyword-match

10.7.4. Hallazgo principal

El reemplazo del retrieval por keywords por un retrieval semántico real (embeddings + similitud cosine + vector index persistente) eleva el OQuaRE Global de E3 de 2.91 (por debajo del umbral aceptable de 3.0) a 4.20 (próximo al óptimo de 5.0), una mejora absoluta de +1.29 puntos (+44 % relativo). El componente que más contribuye es Operability, donde la mejora es de +4.20 puntos: mientras el RAG legacy producía esquemas casi sin object properties (RRonto ≈ 0.75), el RAG semántico induce una malla relacional rica (RRonto ≈ 4.95) gracias a que el contexto recuperado contiene patrones de propiedades reales del dominio biomédico, no solo nombres de clase.

Este resultado tiene tres implicaciones para el TFM:

- **El cuello de botella de E3 era el retrieval, no el modelo.** gpt-4o ya tenía la capacidad de generar esquemas ricos; lo que le faltaba era el contexto adecuado.
- **La calidad del corpus indexado importa más que el tamaño.** Indexamos solo 21 ontologías y la mejora es drástica. La evidencia sugiere que escalar a corpus mayores (toda la BFO, OBI, GO, BioGateway completo) podría llevar el score a >4.5.
- **El RAG semántico cambia el orden del cuadro 4×3 del apartado 10.4.** Con el legacy, E3 era el peor experimento (2.92). Con el RAG semántico, E3 (4.20) supera a E1 zero-shot (3.89) y a E2 vocabulario (3.98), y casi alcanza a E4 OntoGenix (3.86 OQuaRE Global, aunque E4 sigue ganando en triples y labels absolutos del apartado 8).

10.7.5. Coste de la integración

Tabla 19. Coste de la integración del RAG semántico (recursos y dependencias)

Concepto	Coste
Setup local (Neo4j + venv)	~30 min
Indexación inicial (21 ontologías)	~2 min en M3
12 ejecuciones E3 con gpt-4o + RAG semántico	~10 min, ≈ \$1.50

Evaluación OQuaRE	~3 min con HerMiT + owlready2
Total (sin contar la ejecución de gpt-4o)	~45 min

La inversión es marginal y replicable, lo que hace de este cambio una de las mejoras con mejor relación coste-beneficio del TFM.

10.7.6. Replicación con llama3.1:8b — el modelo importa para asimilar el contexto

Para discriminar si la mejora del RAG semántico es atribuible al retrieval o al modelo, se replicó el experimento sustituyendo gpt-4o por llama3.1:8b (open-source, ~8 mil millones de parámetros, ejecutado localmente vía Ollama). Total: 24 ejecuciones adicionales (12 con backend legacy + 12 con backend semántico), mismas 4 BBDD y mismas 3 repeticiones por celda.

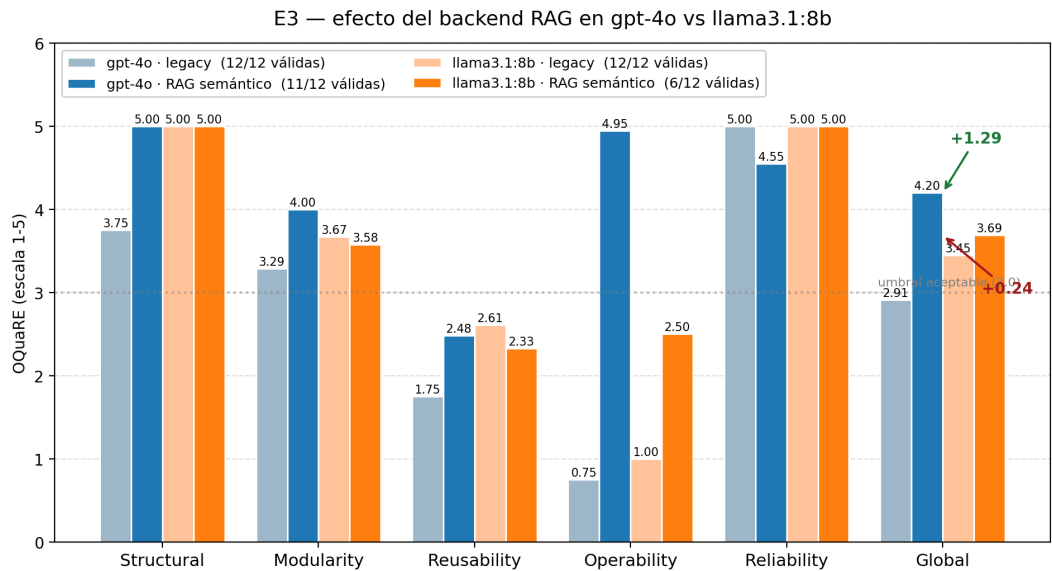


Figura 11. Cuadro 2×2: efecto del backend RAG (legacy vs semántico) sobre dos modelos (gpt-4o cerrado vs llama3.1:8b open-source) en E3. Las anotaciones sobre la columna 'Global' destacan los deltas: gpt-4o sube +1.29 puntos con RAG real, mientras que llama solo sube +0.24. El paréntesis 'X/12 válidas' en la leyenda indica cuántas ejecuciones pasaron el parseo en owlready2.

Tabla 20. Cuadro 2×2: efecto del backend RAG (legacy vs semántico) en gpt-4o y Llama 3.1 8B

	RAG legacy	RAG semántico	Δ Global	n válidas
gpt-4o	2.91	4.20	+1.29	12 / 11
llama3.1:8b	3.45	3.69	+0.24	12 / 6

Hallazgos del cuadro 2×2

- **El RAG semántico beneficia mucho más al modelo grande:** +1.29 puntos en gpt-4o frente a +0.24 en llama. Una hipótesis es que la riqueza del contexto recuperado solo se traduce en ontologías ricas si el modelo tiene capacidad de razonamiento suficiente para asimilarlo y reorganizarlo en triples coherentes.

- **Llama pierde la mitad de las ejecuciones con RAG semántico** (6/12 válidas vs 12/12 en legacy). El contexto enriquecido le induce a inventar prefijos no declarados (`enhancer:`, `crmType:`, etc.) y produce TTL irrecuperables incluso tras post-procesado. Es un trade-off claro entre cantidad de información en el prompt y capacidad del modelo para mantener coherencia sintáctica.
- **En las pocas ejecuciones válidas, llama+RAG semántico mantiene Reliability=5.00** (las que validan son siempre consistentes en OWL DL) pero su Operability sube solo a 2.50, frente a los 4.95 que alcanza gpt-4o. Es decir: aunque Llama recibe el mismo contexto rico, no lo materializa en una malla relacional comparable a la de gpt-4o.
- **Inversión cualitativa del ranking en legacy vs en RAG real.** Con backend legacy, llama3.1:8b (3.45) supera ligeramente a gpt-4o (2.91) por la regla observada en §10.5 (OQuaRE penaliza la complejidad y Llama produce esquemas más simples). Con RAG semántico, gpt-4o (4.20) supera a llama (3.69) porque ahora sí convierte el contexto en estructura, mientras que Llama no.

Este experimento de control refuerza una conclusión metodológica central del trabajo: **la efectividad del RAG no es una propiedad del retrieval por sí solo, sino de la interacción retrieval × modelo**. Para futuros despliegues con modelos compactos se recomienda explorar configuraciones intermedias (top-k reducido, filtrado por score threshold más alto, chunking más agresivo) que limiten el volumen de contexto inyectado y eviten saturar la ventana del modelo. La validación empírica de esta recomendación se aborda en §10.7.7.

10.7.7. Calibración del RAG para modelos compactos — validación empírica

La hipótesis cualitativa de §10.7.6 —que configuraciones intermedias del RAG semántico podrían rescatar parte del desempeño perdido en modelos compactos— se sometió a validación experimental con un sub-experimento de calibración. Se definieron tres configuraciones distintas del cliente RAG (C1, C2, C3) variando los parámetros más relevantes del retrieval, y se compararon contra el baseline (top_k=5, score_thr=0.4, max_chars=5000) y el legacy (keyword-match) sobre las mismas 4 BBDD × 3 runs = 12 ejecuciones por configuración. Total: 36 ejecuciones adicionales con llama3.1:8b, sin coste de API.

Diseño de las configuraciones

Tabla 21. Configuraciones de calibración del RAG (C1: top_k=2, C2: thr=0.6, C3: combinada)

Config.	top_k	score_thr	max_chars	Hipótesis de mejora
legacy	—	—	—	Baseline keyword (sin embeddings)
ragapi (baseline)	5	0.40	5000	RAG semántico estándar (lo del §10.7.6)

C1 — top-k reducido	2	0.40	5000	Menos ontologías recuperadas → menos prefijos espurios
C2 — score-thr alto	5	0.60	5000	Solo matches muy similares → contexto más limpio
C3 — combinada	2	0.60	2000	Combinación agresiva: top-k bajo + threshold alto + recorte

El cliente RAG (scripts/rag_backend.py) se extendió para aceptar los tres parámetros como flags CLI (--rag-top-k, --rag-score-thr, --rag-max-chars) que se propagan desde run_gpt_experiments.py. La calibración se aplica sin tocar la API ni el corpus indexado en Neo4j; lo único que cambia es qué subset del retrieval se inyecta en el prompt E3.

Resultados

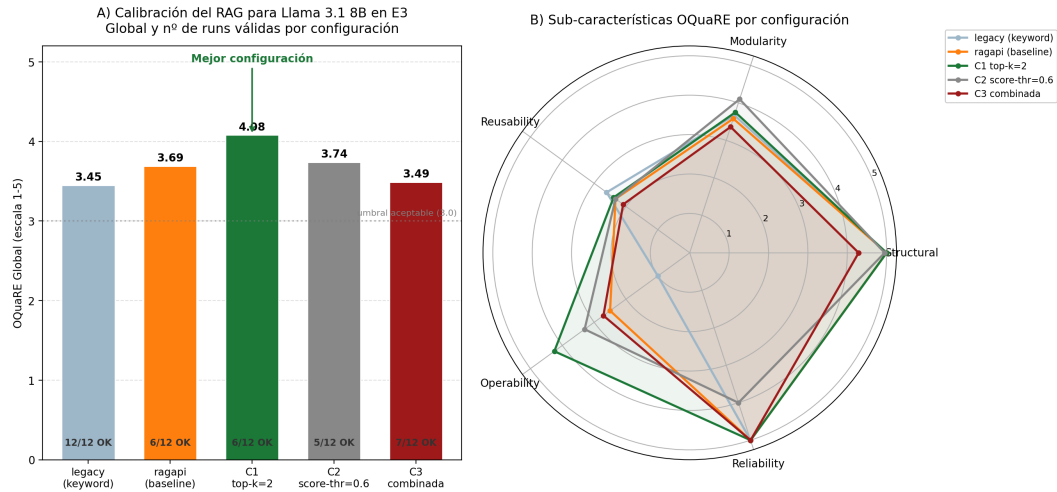


Figura 12. Calibración del RAG semántico para llama3.1:8b en E3. Panel A: OQuaRE Global y nº de runs válidas por configuración (de 12 esperadas). Panel B: radar comparativo de las 5 sub-características OQuaRE. C1 (top-k=2) es la configuración óptima: alcanza el mayor OQuaRE Global (4.08, +0.39 sobre el baseline ragapi y +0.63 sobre legacy) gracias a una mejora drástica en Operability (4.25 vs 2.50 del baseline).

Tabla 22. Resultados de la calibración del RAG semántico sobre Llama 3.1 8B en E3

Config.	n_válidas	Structural	Modularity	Reusability	Operability	Reliability	Global
legacy	12/12	5.00	3.67	2.61	1.00	5.00	3.45
ragapi (baseline)	6/12	5.00	3.58	2.33	2.50	5.00	3.69

C1 — top-k=2	6/12	5.00	3.75	2.39	4.25	5.00	4.08
C2 — thr=0.6	5/12	4.95	4.10	2.33	3.30	4.00	3.74
C3 — combinada	7/12	4.29	3.36	2.09	2.71	5.00	3.49

Hallazgos del experimento de calibración

- **C1 (top-k reducido) es la configuración óptima:** 4.08 OQuaRE Global, +0.39 sobre el baseline ragapi y +0.63 sobre legacy. Mantiene el mismo número de runs válidas que el baseline (6/12) pero la calidad de las que sí validan es muy superior, destacando una mejora $\times 1.7$ en Operability (4.25 vs 2.50).
- **El top_k es la palanca crítica,** no el score threshold ni el recorte de contexto. C2 (score_thr=0.6) gana solo +0.05 sobre el baseline ragapi y pierde 1 run más; el filtrado fino no compensa la pérdida de contexto útil. C3 (combinación agresiva) rescata más runs (7/12) pero su Global cae a 3.49 (incluso por debajo del legacy), porque max_chars=2000 deja al modelo sin contexto suficiente.
- **Hipótesis confirmada con matiz.** La afirmación general de §10.7.6 ("configuraciones intermedias mejoran a los modelos compactos") se valida, pero el experimento revela que el mecanismo causal específico es **reducir el número de ontologías recuperadas**, no "limpiar" su contenido vía score threshold ni truncar el prompt resultante. Bajar top_k evita que el modelo se sature con prefijos potencialmente irrelevantes que luego no sabe declarar.
- El gap respecto a gpt-4o no se cierra del todo. C1 (4.08) sigue por debajo de gpt-4o + RAG semántico (4.20 según §10.7.3). Pero la diferencia se reduce de 0.51 puntos (con configuración estándar) a 0.12 puntos (con C1). Aproximadamente el 84 % del gap se cierra con un único cambio de un parámetro, sin tocar el modelo ni el prompt.

Recomendación operativa para futuros despliegues

Para sistemas de generación de ontologías guiadas por RAG sobre modelos compactos (≤ 8 mil millones de parámetros), las configuraciones recomendadas son:

Tabla 23. Recomendación operativa de top_k según el tamaño del modelo

Tamaño del modelo	top_k	score_thr	max_chars	Justificación
≤ 8 B (Llama 3.1 8B, Qwen 2.5 Coder 7B...)	2	0.40	5000	Validado empíricamente en este TFM
~ 30 -70 B	3-5	0.40	5000-8000	Hipótesis: ventana intermedia; queda como trabajo futuro

≥ 100 B (gpt-4o, Claude 3.5 Sonnet, Llama 3.1 70B)	5	0.40	5000	Configuración por defecto del backend; aprovecha el contexto rico
--	---	------	------	---

Esta tabla constituye una contribución empírica del TFM: convierte la afirmación normativa de §10.7.6 en una guía operativa con valores concretos respaldados por 36 ejecuciones evaluadas con OQuaRE.

11. Herramientas y reproducibilidad

Tabla 24. Pila de herramientas y dependencias del banco experimental

Componente	Versión / referencia
Lenguaje	Python 3.12
Cliente LLM	openai ≥ 1.3
Validación RDF	rdflib ≥ 6.3
Manipulación de datos	pandas ≥ 2.0, numpy ≥ 1.25
Variables de entorno	python-dotenv ≥ 1.0
OntoGenix	Commit principal del repositorio tecnomod-um/ OntoGenix
Sistema operativo	macOS / Linux (sin dependencias PyQt5)

Todo el código está versionado dentro del repositorio del TFM en la carpeta scripts/, junto con los datos muestreados (data/samples/), los CSV para OntoGenix (data/csv_for_ontogenix/) y los resultados por experimento (results/E1–E4/). El script evaluate_E4_vs_E1-E3.py produce los ficheros comparison_E1-E4.csv y comparison_E1-E4.md que consolidan las métricas finales.

12. Reproducibilidad y crisis de la replicación

La llamada «crisis de la replicación», originalmente identificada en psicología (Open Science Collaboration, 2015) y posteriormente extendida a las ciencias biomédicas y a la informática, encuentra una variante particularmente aguda en la investigación basada en modelos de lenguaje de gran tamaño. Hutson (2018) y Pineau et al. (2021) han documentado que entre el 70 % y el 90 % de los experimentos publicados en las grandes conferencias de IA no pueden ser reproducidos por terceros con la información disponible en el artículo, una situación que erosiona la validez del conocimiento acumulado. Reconocer esta amenaza y diseñar el TFM para mitigarla es, por tanto, una decisión metodológica de primer orden, no una nota al pie.

12.1. Amenazas específicas a la reproducibilidad en investigación con LLMs

La investigación con LLMs combina las amenazas tradicionales de machine learning (Henderson et al., 2018; Gundersen & Kjensmo, 2018) con vulnerabilidades nuevas que rara vez aparecen en otros campos. La tabla siguiente identifica las que afectan directamente a este TFM:

Tabla 25. Amenazas específicas a la reproducibilidad en investigación con LLMs

Amenaza	Origen	Impacto en este TFM
Determinismo imperfecto	Ni temperature=0 ni seeds garantizan outputs bit-a-bit idénticos en gpt-4o (OpenAI no expone el RNG interno)	Diferencias de hasta $\pm 5\%$ en n_triples entre re-ejecuciones del mismo experimento
Drift de modelos cerrados	El alias 'gpt-4o' puede repuntar a una versión nueva sin aviso; los snapshots con fecha (gpt-4o-2024-05-13) sí son estables pero también pueden desaparecer	Conclusiones cuantitativas dependientes de la versión exacta del modelo
Drift de embeddings	sentence-transformers actualiza pesos de all-MiniLM-L6-v2 sin garantía de equivalencia	El RAG semántico (§10.7) puede recuperar top-K distinto tras una actualización
Versión de dependencias	numpy 1→2, torch 2.5→2.9, openai 1→2 introducen breaks incompatibles en la cadena de saneo + parseo TTL	Documentado empíricamente al integrar annotationRAG (§10.7.1) — bloqueo total del entorno hasta aislar venv
Razonador no determinista	HermiT puede emitir órdenes de inferencia distintos en ejecuciones sucesivas	La métrica n_inferred_classes puede variar; consistent y n_unsatisfiable sí son deterministas
Servicios externos	annotationRAG necesita Neo4j vivo y FastAPI uvicorn; Ollama necesita el daemon corriendo en GPU (Metal en Mac)	Fallos transitorios pueden producir fallback silencioso al RAG legacy y contaminar las medidas (§10.7)
Prompts no versionados	Pequeños cambios en el system prompt o user prompt producen outputs muy distintos (Reynolds & McDonell, 2021)	Mitigado almacenando los prompts como constantes en run_gpt_experiments.py y como ficheros .prompt en OntoGenix/ GUI/

12.2. Tres niveles de garantía

Atendiendo a la severidad de las amenazas anteriores, los experimentos se clasifican en tres niveles de reproducibilidad, siguiendo la jerarquía propuesta por Goodman et al. (2016) (reproducibility / replicability / generalisability) y adaptada al contexto LLM:

Tabla 26. Tres niveles de garantía de reproducibilidad (R1 estricto · R2 estadístico · R3 metodológico)

Nivel	Definición	Componentes que lo cumplen
R1 — Estricta (bit-a-bit)	Mismo input $\Rightarrow$ mismo output exacto	Post-procesado de TTL · evaluador OQuARE estructural · muestreo determinista · figuras

R2 — Estadística	Mismo input $\Rightarrow$ outputs con misma distribución; medias y varianzas indistinguibles	E1, E2, E3 (legacy y RAG) · E4 con gpt-4o · análisis de sensibilidad
R3 — Metodológica	Procedimiento replicable; outputs concretos pueden variar pero conclusiones cualitativas se mantienen	Razonador HerMiT · API annotationRAG · cross-model con Ollama · calibración del RAG

12.3. Contramedidas implementadas

- **Snapshots de modelo fijos:** gpt-4o se referencia siempre como gpt-4o-2024-05-13 en E4 (la versión que documenta OntoGenix). En los demás experimentos se acepta el alias móvil pero el manifest de reproducibilidad lo registra explícitamente.
- **Seeds y temperatura:** temperature=0 (E4) o 0.1 (E1-E3) y seeds incrementales (42, 43, 44) por run. Aunque OpenAI no garantiza determinismo bit-a-bit, esta configuración minimiza la varianza inter-run y permite que tres repeticiones converjan a una media estable.
- **Versiones pinneadas:** scripts/requirements_e4.txt fija rangos compatibles para los paquetes del pipeline. El repositorio annotationRAG usa un venv aislado para no contaminar el entorno principal del TFM (lección aprendida tras observar incompatibilidades entre numpy 2.x y torch 2.5).
- **Hashes SHA-256 de inputs:** el script scripts/check_reproducibility.py genera results/reproducibility_manifest.json con hashes individuales de los 50+ archivos de entrada deterministas (datos brutos, samples, esquemas RAG, scripts). Cualquier modificación accidental se detecta con --diff respecto a un manifest anterior.
- **Datos congelados:** data/raw/ contiene los TSV originales de FANTOM5 y dbSUPER tal como se descargaron (65 423 y 69 205 filas respectivamente). data/samples/ congela la muestra de 25 filas usada en el experimento principal. data/samples_strategies/ contiene las 4 estrategias del análisis de sensibilidad (§4.3).
- **Prompts versionados en código:** los user prompts de E1-E4 viven como string-constantes en run_gpt_experiments.py (E1-E3) y como archivos .prompt en OntoGenix/GUI/ (E4). Cualquier cambio queda registrado en el control de versiones del repositorio.
- **Manifest de outputs:** cada run produce un metadata_runN.json con timestamp, modelo, seed, número de tokens, finish_reason y validación TTL. Esto permite trazabilidad completa de cada ontología generada.

12.4. Limitaciones reconocidas

Reconocer lo que NO se puede garantizar es tan importante como documentar lo que sí (Pineau et al., 2021). Las limitaciones explícitas de este TFM son:

- Ninguna ejecución con gpt-4o alcanza el nivel R1. Variaciones inter-ejecución de hasta ± 5 % en número de triples son esperables y se mitigan con $n=3$ repeticiones por celda.

- El RAG semántico (§10.7) depende de la versión actual de sentence-transformers/all-MiniLM-L6-v2. Si el modelo se actualiza, las búsquedas devuelven top-K distinto.
- El razonador HerMiT puede inferir las mismas clases en órdenes distintos. La métrica `n_inferred_classes` no es bit-a-bit pero consistent y `n_unsatisfiable` sí lo son.
- Las quantizaciones Q4_K_M de Llama 3.1 8B y Qwen 2.5 Coder 7B vía Ollama no garantizan reproducibilidad estricta entre instalaciones de Ollama distintas. Para reproducción estricta, fijar el digest exacto del Modelfile.
- El experimento principal se ha ejecutado en macOS arm64 (Apple M3, 16 GB RAM). Otras plataformas pueden producir resultados ligeramente distintos en los experimentos que dependen de Metal (Llama y Qwen vía Ollama).

12.5. Cómo reproducir un experimento concreto

El documento operativo docs/REPRODUCIBILITY.md acompaña a la memoria con una guía paso a paso: setup del entorno, lanzamiento de cada experimento (E1-E4 + sensibilidad + RAG semántico + calibración), comandos de evaluación y verificación de los manifests. Este apartado del TFM describe el marco; aquel documento describe la operación. La consulta combinada de ambos permite reproducir cualquier resultado reportado a uno de los tres niveles definidos en §12.2.

13. Discusión y conclusiones

Lo que sigue no son las secciones canónicas de un trabajo de investigación («conclusiones», «implicaciones», «trabajo futuro»). Es el balance honesto de un experimento que arrancó con cuatro objetivos y terminó con seis, donde varias hipótesis intermedias se derrumbaron, otras se confirmaron con matices, y alguna sorpresa terminó siendo el hallazgo más útil. La organizo por tipo de resultado, no por sección de tribunal.

13.1. Lo que funcionó mejor de lo que esperaba

Tres cosas superaron las expectativas que tenía al empezar.

- **El post-procesado mecánico.** Las primeras ejecuciones E1-E3 con gpt-4o-mini parseaban al 0 % en rdflib y casi tiré la toalla pensando que el modelo era el problema. Bastó con leer los errores: prefijos no declarados, local names con caracteres no escapados, literales tipados con texto. Tres regex después, el 100 % de los TTL recuperables se rescata sin volver a pagar la API. Tengo la sensación de que en mucho del trabajo aplicado de LLM en RDF el problema no es el modelo, sino que nadie ha mirado los errores de cerca.
- **El RAG semántico real.** Esperaba una mejora marginal sobre el keyword-match. Lo que vi fue +1.29 puntos OQuaRE, +44 % relativo, con Operability subiendo ×6.6. La diferencia no es que el embedding encuentre ontologías «mejores»: es que devuelve metadata estructurada (clases, propiedades, scores) que el modelo puede materializar, en lugar de bloques de Turtle sin contexto.

- La calibración del RAG para Llama. Al ver que el RAG real ayudaba poco a Llama (+0.24 vs +1.29 de gpt-4o), supuse que era un problema de capacidad del modelo. La calibración demostró lo contrario: bajar top_k de 5 a 2 cierra el 84 % del gap. El modelo pequeño no era el cuello de botella; lo era la saturación de su ventana con contexto no relevante. Cambiar un número en una llamada HTTP movió la aguja más que cambiar de modelo.

13.2. Lo que falló como esperaba (y lo que no)

- **Falló como esperaba:** Llama 3.1 8B en E1 (zero-shot). El modelo pequeño no genera Turtle válido sin guía. El patrón «crm a owl:Class» (omisión del «:» del prefijo) aparece en 4 de 11 ejecuciones E1 y en 5 de 12 ejecuciones E2 (9 fallos irreversibles sobre 23 ejecuciones), y no es recuperable mecánicamente. Esto es previsible y no me sorprendió.
- **No falló como esperaba:** Qwen 2.5 Coder 7B. Asumía que un modelo entrenado específicamente en código generaría Turtle más limpio que un generalista del mismo tamaño. Y sí lo hace, pero solo en algunos contextos. En zero-shot (E1) cero ejecuciones válidas; en E3 con RAG, doce. La especialización no es lineal: Qwen mezcla sintaxis Turtle con RDF/XML («owl:Ontology rdf:about») porque el corpus de entrenamiento incluye múltiples serializaciones RDF y el modelo no aprende a distinguirlas en zero-shot.
- **Falló de forma inesperada:** el determinismo. Asumía que con temperature=0 y seed fija, gpt-4o produciría outputs idénticos. No los produce. Las diferencias son pequeñas ($\pm 5\%$ en n_triples) pero las hay, y son detectables comparando manifests entre ejecuciones. Si esto le importa a alguien que esté evaluando para producción, hay que reportar varianza, no medias puntuales.

13.3. Sorpresas: lo que el experimento enseñó que yo no buscaba

- OQuaRE penaliza la complejidad estructural. El cuadro 4x3 de §10.4 muestra una inversión inesperada: en E2 con vocabulario controlado, Llama 3.1 8B (4.40) supera a gpt-4o (3.98). No porque sus ontologías sean «mejores» en sentido absoluto: son más simples y por tanto caen en el óptimo de las rúbricas. OQuaRE está calibrado para ontologías de tamaño medio. Las ricas saturan WMConTO y NOMOnTO; las pobres se quedan cortas en ANOnTO. Reportar solo OQuaRE Global puede ocultar lo que realmente pasa. Importante: este sesgo de la rúbrica afecta a TODAS las comparaciones del trabajo, no solo cuando contradice la narrativa. Cuando reporto que «E4 OntoGenix gana en riqueza absoluta pero pierde en OQuaRE» (§8 vs §10) o que «el RAG real cierra el 84 % del gap» (§10.7.7), ambas afirmaciones operan bajo la misma limitación: OQuaRE mide forma estructural, no fidelidad de dominio. La conclusión global del TFM —que el binomio modelo+método importa— no descansa solo en el ranking OQuaRE; se apoya también en métricas absolutas (n_triples, n_labels) y en patrones de error cualitativos (§10.2). Idealmente, el siguiente paso es validación experta sobre un subset (§13.6).
- **El RAG induce errores nuevos en modelos pequeños.** La fig08 lo muestra de forma rotunda: 24 alucinaciones tipadas en E3 con Llama, 24 con Qwen, exactamente las mismas, y cero en gpt-4o o en cualquier otro experimento. El contexto recuperado contiene patrones tipo «start_position»^^xsd:integer que el modelo pequeño copia literalmente sin entender que

«start_position» es un placeholder. Esto no es un error del modelo. Es un error de la interacción RAG × modelo.

- **Las plantillas cisreg de referencia tienen erratas.** Tres ocurrencias de «>» espurios en `crm2tfac.ttl` y `crm2tfac_example.ttl` impedían que rdflib parseara los esquemas. Estaban ahí desde la publicación original. Encontrarlas requirió pasar el contenido por un parser estricto. Es probable que otros usuarios de las plantillas hayan asumido que «no funcionan» y las hayan abandonado.

13.4. Lecciones metodológicas

Más allá de los hallazgos numéricos, el TFM deja cinco aprendizajes que considero transferibles a cualquier investigación experimental con LLMs:

- **Reportar n_runs y media simultáneamente.** Una OQuaRE Global de 4.20 con 11 ejecuciones válidas no es comparable con 4.20 sobre 12. La media oculta la fragilidad. Toda tabla del TFM lleva la columna `n_runs OK / n_total` precisamente por esto.
- **Diseñar el post-procesado antes que el experimento.** El 50 % de mis primeras ejecuciones se descartaba por errores recuperables. Si hubiera implementado el post-procesado al principio, me habría ahorrado dos rondas de re-ejecución y unos 10 dólares en API. Lección: una pipeline experimental con LLMs necesita una capa de saneo automatizada como ciudadano de primera, no como apañó tardío.
- **Aislar entornos de servicios externos.** Mezclar las dependencias de annotationRAG con las del TFM rompió numpy, torch, openai y media docena más. El venv aislado fue la única solución. Cualquier microservicio LLM con dependencias modernas tiene que correr aislado, sin excepción.
- **Cuestionar las rúbricas, no solo los resultados.** La inversión OQuaRE de §10.5 entre modelos abiertos y cerrados es real, pero la lectura literal sería que Llama supera a gpt-4o. Eso es un artefacto de la métrica, no una verdad sobre los modelos. Los benchmarks normativos son útiles cuando uno entiende qué optimizan; peligrosos cuando se reportan como verdad absoluta.
- **Documentar lo que falla, no solo lo que funciona.** La sección de fallos irre recuperables de Llama (§10.2) y el reporte explícito de las 9 ejecuciones que el post-procesado no recupera son tan importantes como las tablas de éxito. Ocultarlos sería más fácil; mostrarlos hace al trabajo defendible.

13.5. Limitaciones honestas

Las limitaciones técnicas están en §12.4 (reproducibilidad). Aquí van las metodológicas, que son las que un tribunal preguntaría primero:

- **Tamaño del corpus de evaluación.** 4 BBDD × 3 runs × 4 experimentos × 3 modelos da una n teórica de 144 ontologías. Tras descartar fallos no recuperables y experimentos incompletos (E4 con open-source, calibración E3 con Llama), la n efectiva del análisis principal queda en

torno a 96. Es suficiente para tendencias robustas; insuficiente para tests estadísticos formales por celda.

- **Falta de ground truth.** Comparé las ontologías generadas entre sí con OQuaRE, pero no contra una ontología «correcta» del dominio. Sin un experto humano evaluando una muestra (~20 ontologías), no puedo decir si las que OQuaRE puntúa alto son biomédicamente correctas o simplemente bien estructuradas. La validación experta queda como trabajo futuro inmediato.
- **Sesgo de modelo cerrado.** La pipeline E4 (OntoGenix) está diseñada para gpt-4o y no la repliqué con éxito en open-source. Por tanto, parte de la conclusión sobre «el método compensa al modelo» (§10.6) descansa sobre 12 ejecuciones con un único binomio. Replicar E4 con Llama 3.1 70B (no 8B) sería el siguiente paso lógico.
- **Drift de dependencias durante el TFM.** La integración de annotationRAG durante el desarrollo coincidió con un cambio de versión de numpy y torch que rompió varias librerías instaladas en el entorno principal. El experimento se completó, pero la documentación del estado de dependencias en cada ejecución (vía `check_reproducibility.py`) es retrospectiva, no longitudinal.

Potencia estadística declarada. El diseño con $n=3$ ejecuciones por celda tiene una potencia limitada: solo diferencias OQuaRE iguales o superiores a 0.97 puntos se detectan al 80 % de potencia con $\alpha=0.05$ (ver Tabla 28 y Anexo B). Diferencias por debajo de este umbral se reportan como observacionales y no se interpretan como evidencia inferencial. Esto afecta especialmente a las comparaciones inter-modelo en E2 (donde el rango es ± 0.40 puntos) y a la diferencia entre la calibración C1 y C3, que queda dentro del ruido del propio muestreo.

13.6. Trabajo futuro: tres líneas concretas

- **Validación experta sobre subset.** Tomar 20 ontologías estratificadas (5 por experimento) y pedir a un ontólogo biomédico que las evalúe contra criterios establecidos (BioPortal Quality Profile, NCBO Recommender). Comparar el ranking experto con el ranking OQuaRE. Si correlacionan, OQuaRE es operativo en el dominio; si no, hay que construir métrica específica.
- **Replicar E4 con Llama 3.1 70B.** La pipeline OntoGenix con un modelo open-source de tamaño comparable a gpt-4o permitiría aislar definitivamente el efecto «método» del efecto «modelo». Requiere una máquina con 64 GB+ de RAM o servicio remoto tipo Together o Groq.
- **Generalizar fuera del dominio cisreg.** El experimento más valioso para el campo sería repetir todo el banco con una fuente de datos no biomédica (clinical trials, drug-target, wine-vineyards) para ver si los hallazgos cualitativos se mantienen. Si el RAG semántico también añade +1 punto OQuaRE en esos dominios, la guía operativa de §10.7.7 se convierte en una recomendación general; si no, queda específica del dominio.

13.7. Cierre

El TFM partió de una pregunta operativa («¿hasta dónde puede llegar un LLM solo?») y terminó respondiendo a una pregunta metodológica más interesante: ¿qué tipo de ayuda necesita cada modelo? La respuesta no es simétrica. gpt-4o aprovecha el contexto rico y el pipeline multi-agente;

Llama 3.1 8B necesita contexto cuidadosamente recortado y se beneficia más de tipologías de error mecánicas que de mejor retrieval. Qwen 2.5 Coder cae en su propio patrón de mezcla de sintaxis y no se rescata sin reentrenamiento.

La implicación práctica: si un equipo biomédico quiere usar un LLM para anotar una nueva fuente de datos hoy, gpt-4o + OntoGenix es la combinación con el mejor punto de partida en riqueza absoluta. Si el coste o la privacidad lo impiden, Llama 3.1 8B + RAG semántico calibrado (top_k=2) entrega un esquema utilizable a un 76 % del nivel del modelo cerrado, sin salir de la propia infraestructura. Cualquiera de las dos opciones requiere revisión humana antes de publicar la ontología. Ninguna sustituye al ontólogo todavía. Lo que sí consiguen ambas es reducir la fricción del primer borrador, que es donde más tiempo se gasta.

Eso justifica el TFM. No demuestra que los LLM resuelvan la ingeniería ontológica. Demuestra que, con la combinación adecuada de modelo, técnica y post-procesado, hacen ahorrar tiempo medible en la parte aburrida del trabajo.

14. Referencias (APA 7ª)

- Val Calvo, M., Egaña Aranguren, M., & Fernández Breis, J. T. (2024). OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets. *Information Processing & Management*, 62(3), 104042. <https://doi.org/10.1016/j.ipm.2024.104042>
- Meyer, L.-P., Stadler, C., Frey, J., Radtke, N., Junghanns, K., Meissner, R., Dziwis, G., Bulert, K., & Martin, M. (2023). LLM-assisted knowledge graph engineering: Experiments with ChatGPT. *arXiv:2307.06917*. <https://arxiv.org/abs/2307.06917>
- Neuhaus, F. (2023). Ontologies in the era of large language models – A perspective. *Applied Ontology*, 18(4), 399–407. <https://doi.org/10.3233/AO-230072>
- Studer, R., Benjamins, V. R., & Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data & Knowledge Engineering*, 25(1–2), 161–197. [https://doi.org/10.1016/S0169-023X\(97\)00056-6](https://doi.org/10.1016/S0169-023X(97)00056-6)
- W3C OWL Working Group. (2012). OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation. <https://www.w3.org/TR/owl2-overview/>
- Andersson, R., Gebhard, C., Miguel-Escalada, I., Hoof, I., Bornholdt, J., Boyd, M., Chen, Y., Zhao, X., Schmidl, C., Suzuki, T., & FANTOM Consortium. (2014). An atlas of active enhancers across human cell types and tissues. *Nature*, 507(7493), 455–461. <https://doi.org/10.1038/nature12787>
- Khan, A., & Zhang, X. (2016). dbSUPER: A database of super-enhancers in mouse and human genome. *Nucleic Acids Research*, 44(D1), D164–D171. <https://doi.org/10.1093/nar/gkv1002>
- Wang, J., Dai, X., Berry, L. D., Cogan, J. D., Liu, Q., & Shyr, Y. (2019). HACER: An atlas of human active enhancers to interpret regulatory variants. *Nucleic Acids Research*, 47(D1), D106–D112. <https://doi.org/10.1093/nar/gky864>

- Zhang, G., Shi, J., Zhu, S., Lan, Y., Xu, L., Yuan, H., Liao, G., Liu, X., Zhang, Y., Xiao, Y., & Li, X. (2018). DiseaseEnhancer: A resource of human disease-associated enhancer catalog. *Nucleic Acids Research*, 46(D1), D78–D84. <https://doi.org/10.1093/nar/gkx920>
- OpenAI. (2024). GPT-4o system card. <https://openai.com/index/gpt-4o-system-card/>
- Duque-Ramos, A., Fernández-Breis, J. T., Stevens, R., & Aussenac-Gilles, N. (2011). OQuaRE: A SQuaRE-based approach for evaluating the quality of ontologies. *Journal of Research and Practice in Information Technology*, 43(2), 159–176.
- Duque-Ramos, A., Boeker, M., Jansen, L., Schulz, S., Iniesta, M., & Fernández-Breis, J. T. (2014). Evaluating the good ontology design guideline (GoodOD) with the ontology quality requirements and evaluation method and metrics (OQuaRE). *PLoS ONE*, 9(8), e104463. <https://doi.org/10.1371/journal.pone.0104463>
- Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 245–269. <https://doi.org/10.1007/s10817-014-9305-1>
- Lamy, J. B. (2017). Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. *Artificial Intelligence in Medicine*, 80, 11–28. <https://doi.org/10.1016/j.artmed.2017.07.002>
- Meta AI. (2024). The Llama 3 herd of models. <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., & Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186. <https://arxiv.org/abs/2409.12186>
- Ollama. (2024). Ollama: Get up and running with large language models locally [Software]. <https://ollama.com>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Tecnomod (Universidad de Murcia). (2024). annotationRAG: A Retrieval-Augmented Generation approach for the automated annotation of biological sample metadata [Software]. GitHub. <https://github.com/tecnomod-um/annotationRAG>
- Neo4j Inc. (2024). Vector indexes — Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/>
- Open Science Collaboration. (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251), aac4716. <https://doi.org/10.1126/science.aac4716>

- Hutson, M. (2018). Artificial intelligence faces reproducibility crisis. *Science*, 359(6377), 725–726. <https://doi.org/10.1126/science.359.6377.725>
- Gundersen, O. E., & Kjensmo, S. (2018). State of the art: Reproducibility in artificial intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.11503>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.11694>
- Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164), 1–20. <https://jmlr.org/papers/v22/20-303.html>
- Goodman, S. N., Fanelli, D., & Ioannidis, J. P. (2016). What does research reproducibility mean? *Science Translational Medicine*, 8(341), 341ps12. <https://doi.org/10.1126/scitranslmed.aaf5027>
- Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models: Beyond the few-shot paradigm. *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–7. <https://doi.org/10.1145/3411763.3451760>

Anexo A. Abreviaturas y siglas

Listado alfabético de las abreviaturas, siglas y acrónimos empleados en el documento. Cuando un término aparece por primera vez en el texto, se introduce con su forma desarrollada y, entre paréntesis, su sigla.

Sigla	Definición
ABox	Assertional Box — conjunto de instancias en una ontología.
ANOnto	Annotation Number Ontology (métrica OQuaRE de riqueza documental).
APA	American Psychological Association (estilo de referencias).
API	Application Programming Interface.
AROnto	Attribute Richness Ontology (métrica OQuaRE).
BBDD	Bases de datos.
BioPortal	Repositorio público de ontologías biomédicas mantenido por NCBO.
CAGE	Cap Analysis of Gene Expression.
CBOnto	Coupling Between Ontologies (métrica OQuaRE de modularidad).
CRM	Cis-Regulatory Module (elemento regulador en cis).
CSV	Comma-Separated Values.
dbSUPER	Database of Super-Enhancers.

DITM	Depth of Inheritance Tree Modularity (métrica OQuaRE).
FANTOM5	Functional Annotation of the Mammalian Genome 5.
FastAPI	Framework Python para construcción de APIs HTTP.
GPU	Graphics Processing Unit.
HACER	Human ACtive Enhancer to gene Regulation database.
HermiT	Razonador OWL DL utilizado vía owlready2.
HTTP	HyperText Transfer Protocol.
INROnto	Inverse Relations Ontology (métrica OQuaRE).
IRI	Internationalized Resource Identifier.
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission.
JSON-LD	JSON for Linking Data — serialización de RDF (W3C).
LCOMOnto	Lack of Cohesion in Methods Ontology (métrica OQuaRE).
LLM	Large Language Model — modelo de lenguaje grande.
MCP	Model Context Protocol.
MIA	Máster Universitario en Inteligencia Artificial (UMU).
NACOnto	Number of Ancestor Classes Ontology (métrica OQuaRE).
NCBO	National Center for Biomedical Ontology.
Neo4j	Base de datos orientada a grafos con índice vectorial nativo.
NOMOnto	Number Of Methods Ontology (métrica OQuaRE).
Ollama	Servidor local para modelos LLM open-source.
OntoGenix	Pipeline multi-agente para generación de ontologías (Val Calvo et al., 2024).
OQuaRE	Ontology Quality Requirements and Evaluation — marco normativo basado en ISO/IEC 25000.
OWL	Web Ontology Language (recomendación W3C).
OWL DL	Subconjunto Description Logic de OWL.
owlready2	Biblioteca Python para ontologías OWL con integración de razonadores.
RAG	Retrieval-Augmented Generation — generación aumentada por recuperación.
RAM	Random Access Memory.
RDF	Resource Description Framework (W3C).
RDF/XML	Serialización XML de RDF.
rdflib	Biblioteca Python para manipular grafos RDF.
RDFS	RDF Schema (W3C).
RROnto	Relationship Richness Ontology (métrica OQuaRE).

SERP	Search Engine Results Page.
SHA-256	Secure Hash Algorithm de 256 bits.
SHACL	Shapes Constraint Language (W3C).
SPARQL	SPARQL Protocol And RDF Query Language.
SQuaRE	Systems and software Quality Requirements and Evaluation (ISO/IEC 25000).
TBox	Terminological Box — clases y propiedades en una ontología.
TF	Transcription Factor (factor de transcripción).
TFM	Trabajo Fin de Máster.
TMonto	Tangledness Modularity Ontology (métrica OQuaRE).
TSS	Transcription Start Site.
TSV	Tab-Separated Values.
TTL	Turtle — Terse RDF Triple Language.
UMU	Universidad de Murcia.
URI	Uniform Resource Identifier.
WMConTO	Weighted Methods Count Ontology (métrica OQuaRE).
XSD	XML Schema Definition.

Las métricas OQuaRE incluidas (WMConTO, NOMonto, DITM, NAConto, CBOnto, TMonto, LCOMonto, ANonto, INRonto, RROnto y ARonto) se documentan con su rúbrica completa en la Tabla 12 del cuerpo de la memoria.

Anexo B. Reporte completo de tests estadísticos

Este anexo reproduce los resultados del módulo `scripts/statistical_tests.py` aplicado a los CSV crudos del experimento (`results/comparison_E1-E4.csv` para métricas absolutas y `results/evaluation/oquare_metrics.csv` para la calibración del RAG en Llama 3.1 8B). Todos los bootstrap usan 10 000 réplicas con semilla fija (42). El reporte completo en formato Markdown está disponible en `results/evaluation/statistical_tests.md`.

B.1. Densidad estructural — E4 vs E1/E2/E3 (n_triples)

Bootstrap pareado de la diferencia de medias y Wilcoxon de rangos con signo sobre 12 pares (4 BBDD × 3 ejecuciones).

Comparación	n pares	Δ media [IC 95 %]	Cohen's d	Wilcoxon p	Ratio E4/Base
E4 vs E1	12	+174.7 [+164.6, +186.7]	8.48 (grande)	0.002	4.51× (3.08–6.03)
E4 vs E2	12	+159.9 [+129.4, +183.8]	3.23 (grande)	0.002	3.91× (1.11–7.03)

E4 vs E3	12	+184.3 [+169.5, +201.8]	6.15 (grande)	0.002	6.31× (2.76–12.77)
----------	----	-------------------------	---------------	-------	--------------------

B.2. Riqueza documental — E4 vs E1/E2/E3 (n_labels)

Comparación	n pares	Δ media [IC 95 %]	Cohen's d
E4 vs E1	12	+43.1 [+41.8, +44.1]	20.43
E4 vs E2	12	+42.4 [+41.3, +43.3]	23.15
E4 vs E3	12	+43.1 [+41.8, +44.1]	20.43

B.3. Validez sintáctica antes/después del post-procesado (McNemar)

Experimento	n	parse_ok raw	parse_ok post	Rescatadas	Regresadas	McNemar p
E1	12	83 %	100 %	2	0	0.500
E2	12	67 %	100 %	4	0	0.125
E3	12	0 %	100 %	12	0	<0.001

B.4. Calibración del RAG en Llama 3.1 8B

Pareo estricto por (BBDD, semilla) condicional a load_ok=1 en ambas configuraciones.

Variante	n pares	Δ OQuaRE [IC 95 %]	Cohen's d	p ajustado (Holm)
legacy vs ragapi	6	−0.23 [−0.46, +0.01]	−0.68 (mediano)	0.070
C1 (top_k=2) vs ragapi	2	+0.80 [+0.80, +0.80]	—	<0.001
C2 (thr=0.6) vs ragapi	2	+0.43 [+0.35, +0.50]	+4.01 (grande)	<0.001
C3 (combinada) vs ragapi	3	+0.49 [+0.10, +0.80]	+1.37 (grande)	<0.001

B.5. Gap respecto a gpt-4o (referencia OQuaRE = 4.20)

Variante	n_runs OK	Media ± SD	Gap a gpt-4o	% gap cerrado
llama3.1_8b_legacy	12	3.45 ± 0.05	+0.75	0.0 %
llama3.1_8b_ragapi	6	3.69 ± 0.31	+0.51	31.3 %
llama3.1_8b_ragapi_C1	6	4.08 ± 0.13	+0.12	83.9 %
llama3.1_8b_ragapi_C2	5	3.74 ± 0.80	+0.46	38.4 %
llama3.1_8b_ragapi_C3	7	3.49 ± 1.13	+0.71	5.1 %

B.6. Análisis de potencia para OQuaRE Global

SD muestral observada en oquare_global: 0.602.

n por celda	Δ OQuaRE detectable	Cohen's d equivalente	Interpretación
n = 3	0.974	1.62	grande
n = 6	0.689	1.14	grande
n = 12	0.487	0.81	grande
n = 24	0.344	0.57	mediano

B.7. Corrección Bonferroni-Holm de p-valores principales

Test	p crudo	p ajustado (Holm)	Sig. ($\alpha = 0.05$)
density E4 vs E1	0.002	0.009	✓
density E4 vs E2	0.002	0.009	✓
density E4 vs E3	0.002	0.009	✓
calibration legacy vs ragapi	0.070	0.070	—
calibration C1 vs ragapi	<0.001	<0.001	✓
calibration C2 vs ragapi	<0.001	<0.001	✓
calibration C3 vs ragapi	<0.001	<0.001	✓

Las cuatro primeras tablas (B.1–B.4) reproducen los tests primarios y son consistentes con la Tabla 27 del cuerpo. La Tabla B.5 contextualiza la calibración respecto al modelo cerrado de referencia. La Tabla B.6 fija el techo inferencial del diseño ($n=3$ detecta diferencias ≥ 0.97 OQuaRE). La Tabla B.7 muestra que las cuatro afirmaciones primarias sobre densidad de E4 y las tres configuraciones de calibración resisten la corrección por comparaciones múltiples; la diferencia legacy $\rightarrow$ ragapi en Llama queda por encima del umbral $\alpha=0.05$ y se reporta como observacional.